```
#!/usr/bin/env python3
"""
INDIA MOAT SCREENER v2
======================
What it does
  1. Pulls candidates from Screener.in (growth + quality), then enforces YOUR rul
       - all-time-high quarterly sales   - all-time-high quarterly profit
       - all-time-high annual sales      - all-time-high annual profit
       - profit BREAKOUT (qtr profit >= +25% YoY and above last 4 quarters)
  2. Adds live market data (Yahoo Finance): price vs 50/200 DMA, 52-wk high, volu
  3. Adds news for every shortlisted stock (Google News RSS) with positive/negati
  4. Works out the cycle phase of each theme (precision, T&D, chemicals, semis, r
  5. Scores every stock 0-100, says Bullish / Watch / Avoid and WHY, with stop-lo
  5b. Finds stocks where the 50-DMA and 200-DMA are meeting (golden cross / squee
      debtor days and inventory days are NOT rising (working-capital health).
  6. Builds a mobile-friendly dashboard (docs/index.html) and sends Telegram/emai

Modes
  python auto_screener.py --mode full   # daily: screen + fundamentals + everythi
  python auto_screener.py --mode live   # intraday: refresh price/volume/news on
  python auto_screener.py --mode demo   # fake data, to preview the dashboard

Env vars: SCREENER_USER, SCREENER_PASS  (full mode)
          TELEGRAM_TOKEN, TELEGRAM_CHAT_ID  (optional alerts)
          EMAIL_USER, EMAIL_PASS, EMAIL_TO  (optional, Gmail app password)
Educational screening aid - NOT investment advice.
"""
import os, re, io, sys, json, time, hashlib, smtplib, argparse, random
import statistics as st
from datetime import datetime, timezone, timedelta
from email.message import EmailMessage
from email.utils import parsedate_to_datetime
import xml.etree.ElementTree as ET
import requests
from bs4 import BeautifulSoup

# ---------------------------------------------------------------------------
# CONFIG - edit freely
# ---------------------------------------------------------------------------
BASE = "https://www.screener.in"
OUT_DIR = os.getenv("OUT_DIR", "site_out")
DATA_DIR = f"{OUT_DIR}/data"
DATA_FILE = f"{DATA_DIR}/state.json"
```

```
UA = {"User-Agent": "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 Chrome/12
IST = timezone(timedelta(hours=5, minutes=30))

# Wide net on Screener; the strict ATH / breakout rules are enforced in Python be
CANDIDATE_QUERY = """Market Capitalization > 1000 AND
Sales growth 3Years > 12 AND
Profit growth 3Years > 15 AND
Return on capital employed > 15 AND
Debt to equity < 1"""

MAX_PAGES = 40
BREAKOUT_PROFIT_YOY = 25      # % qtr profit growth YoY for "breakout"
NEAR_HIGH_PCT = 5             # within x% of 52-wk high = price breakout zone
MIN_TURNOVER_CR = 2           # avg daily traded value (Rs Cr) below this = illiq
ALERT_MIN_SCORE = 65
LIVE_SHORTLIST_MAX = 60
ENRICH_MAX = 40               # how many top stocks get concall / order-book / cr
MAX_CANDIDATES = 600          # safety cap on how many Screener results are analy
# Symbols (as on Screener.in, e.g. POLYCAB) that are ALWAYS tracked and searchabl
# You can also add symbols without editing code: Actions > Run workflow > "watch"
EXTRA_WATCHLIST = []
FILING_KW = ["receipt of order", "bagging", "bags order", "order worth", "letter
             "new order", "supply order", "preferential", "warrant", "qip", "righ
             "stake", "acquisition", "subscription", "buyback", "bulk deal", "blo
# Company-size bands in Rs Cr of market cap (edit if you prefer other cut-offs).
CAP_BANDS = [("Micro cap", 0, 500), ("Small cap", 500, 5000), ("Mid cap", 5000, 2
# Areas shown in the "Future tech" tab (names must match AREAS keys). Add or remo
FUTURE_AREAS = ["Semiconductors", "Electronics & EMS", "Data centers & cooling",
                "Solar & renewables", "Green hydrogen & fuel cells", "Nuclear", "
                "Advanced materials & critical minerals", "Telecom, 5G & fibre",
                "Industrial automation & capital goods"]
# Supply-chain chokepoints (my own judgement - edit freely). The "Bottlenecks" ta
# and your loaded stocks that work in the listed areas (names must match AREAS ke
BOTTLENECKS = [
    {"id": "grid", "title": "Power grid: solar can generate, but storage and the
     "chain": "Solar and wind  >  battery storage (BESS)  >  transmission lines
     "why": "Renewable capacity is being added faster than the grid can carry or
     "look": "Order backlog above 1.5x sales, capacity expansion, lead-time comme
     "areas": ["Transmission, transformers & T&D", "Solar & renewables", "EV & ba
     "queries": ["India grid transmission bottleneck renewable curtailment", "bat
    {"id": "minerals", "title": "Tungsten, rare earths and other minerals China h
     "chain": "Mines and concentrate  >  refining (tungsten powder, carbide, anti
     "why": "China dominates the processing of tungsten, antimony, gallium, germa
     "look": "Approvals from defence or aerospace customers, new refining or powd
     "areas": ["Advanced materials & critical minerals", "Mining & minerals", "De
     "queries": ["China export controls tungsten India", "rare earth magnets Indi
```

```
    {"id": "energetics", "title": "Defence ammunition and energetics: shells, pro
     "chain": "Chemical inputs  >  propellants and explosives  >  shells and fuze
     "why": "Ammunition stockpiles are being rebuilt worldwide, and India is cutt
     "look": "Export orders, Ministry of Defence contracts, capacity additions, w
     "areas": ["Ammunition & explosives", "Defence", "Specialty chemicals"],
     "queries": ["ammunition shortage artillery shells production", "India ammuni
    {"id": "aero", "title": "Aerospace castings, forgings and engine parts",
     "chain": "Raw alloys  >  castings and forgings  >  precision machining  >  e
     "why": "Aircraft and engine makers have long backlogs and struggle to get ca
     "look": "Named global customers, long-term agreements, certifications, custo
     "areas": ["Aerospace", "Precision engineering & components", "Space"],
     "queries": ["aerospace supply chain castings forgings shortage", "Boeing Air
    {"id": "electronics", "title": "Electronics and semiconductors: components an
     "chain": "Chips and passives  >  PCBs, connectors, camera and display module
     "why": "Phone and electronics makers want a second source outside China. Ind
     "look": "Customer approvals (Apple, Samsung and their vendors), PLI or schem
     "areas": ["Electronics & EMS", "Semiconductors", "Telecom, 5G & fibre"],
     "queries": ["India electronics component manufacturing scheme approval", "se
    {"id": "datacentre", "title": "Data centres: power, cooling and electrical eq
     "chain": "Grid connection  >  transformers and switchgear  >  UPS and backup
     "why": "AI data centres need huge amounts of power, cooling and backup. Tran
     "look": "Data-centre order inflow, cooling and power-gear capacity, customer
     "areas": ["Data centers & cooling", "Transmission, transformers & T&D", "Rob
     "queries": ["data centre power shortage India", "AI data center cooling dema
    {"id": "nuclear", "title": "Nuclear: heavy forgings, fuel and very few approv
     "chain": "Fuel and heavy water  >  reactor-grade forgings  >  steam generato
     "why": "India plans a large nuclear expansion and is opening the sector to m
     "look": "Approved-vendor status, orders from the nuclear power operator and
     "areas": ["Nuclear", "Precision engineering & components"],
     "queries": ["India nuclear power expansion orders vendors", "NPCIL order hea
    {"id": "copper", "title": "Copper, aluminium and cable inputs",
     "chain": "Mines  >  smelting and refining  >  wire, cable and busbar  >  gri
     "why": "Grid, EV and data-centre build-outs use huge amounts of copper and a
     "look": "Cost pass-through, order books, capacity additions.",
     "areas": ["Mining & minerals", "Transmission, transformers & T&D"],
     "queries": ["copper shortage price record grid demand", "India copper smelte
    {"id": "shipbuilding", "title": "Shipbuilding and marine: global yard capacit
     "chain": "Steel and engines  >  shipyards  >  naval and commercial ships  >
     "why": "Yard capacity is short worldwide, order books run for years, and Ind
     "look": "Order book in years of sales, naval versus commercial mix, margins.
     "areas": ["Shipbuilding & marine", "Defence"],
     "queries": ["global shipbuilding capacity shortage order backlog", "India sh
]
ONLY_THEMES = False   # False = scan ALL sectors (your 5 priority themes are stil

# keyword -> matched against Screener industry tags + company name
```

```
THEMES = {
    "Precision / Aerospace / Defence": ["aerospace", "defence", "defense", "preci
    "Transformers / T&D / Power equipment": ["transformer", "electrical equipment
    "Specialty / Water chemicals": ["specialty chem", "chemical", "water", "agroc
    "Semiconductor / Electronics": ["semiconductor", "electronic", "ems", "power
    "Rail / Automation / Industrial": ["railway", "rail", "automation", "industri
}

THEME_INFO = {
    "Precision / Aerospace / Defence": {
        "drivers": "Defence indigenisation + export push, global aerospace OEM su
        "risks": "Lumpy orders, government payment cycles, premium valuations."},
    "Transformers / T&D / Power equipment": {
        "drivers": "Grid expansion, renewable evacuation, data-centre/industrial
        "risks": "Copper/CRGO input costs, new capacity from many players, valuat
    "Specialty / Water chemicals": {
        "drivers": "China+1 shift, regulatory approvals and sticky customers, wat
        "risks": "Chinese dumping, pricing cycles, destocking, environmental comp
    "Semiconductor / Electronics": {
        "drivers": "ISM/PLI subsidies, EMS and component localisation, electronic
        "risks": "Long gestation, thin margins in assembly, subsidy dependence, v
    "Rail / Automation / Industrial": {
        "drivers": "Rail capex, Kavach/modernisation, factory automation, manufac
        "risks": "Government order timing, working-capital stretch, cyclicality."
}

POS_WORDS = ["order win", "bags order", "wins order", "secures order", "order boo
             "record", "capacity expansion", "expansion", "upgrade", "target pric
             "partnership", "profit jumps", "profit rises", "profit surges", "sur
             "acquisition", "commission", "beats", "raises guidance", "buyback",
NEG_WORDS = ["probe", "sebi", "fraud", "downgrade", "resign", "pledge", "loss wid
             "penalty", "raid", "default", "cuts guidance", "misses", "delist", "
             "fire at", "shutdown", "promoter sells", "stake sale", "insolvency",

# ---- PLI / INCENTIVE SCHEMES (approximate outlays from public announcements; ve
PLI_SCHEMES = [
    {"scheme": "Mobile phones and specified electronic components (PLI)", "outlay
     "why": "Turn India into a global phone/electronics export base and cut impor
    {"scheme": "Electronics Component Manufacturing Scheme (ECMS)", "outlay_cr":
     "why": "Build local PCBs, passive parts and sub-assemblies so assemblers sto
    {"scheme": "India Semiconductor Mission (fabs, OSAT, display)", "outlay_cr":
     "why": "Strategic chip self-reliance; fiscal support of up to about half of
    {"scheme": "IT hardware PLI 2.0 (laptops, servers)", "outlay_cr": 17000, "kw"
     "why": "Localise laptop, tablet and server production."},
    {"scheme": "Automobiles and auto components", "outlay_cr": 25938, "kw": ["aut
     "why": "Encourage advanced-technology vehicles (EV, hydrogen) and local comp
```

```
    {"scheme": "Advanced Chemistry Cell (ACC) battery", "outlay_cr": 18100, "kw":
     "why": "Create 50 GWh of domestic battery cell capacity for EVs and grid sto
    {"scheme": "High-efficiency solar PV modules", "outlay_cr": 24000, "kw": ["so
     "why": "Integrated solar manufacturing (polysilicon to modules) to cut Chine
    {"scheme": "Pharmaceuticals", "outlay_cr": 15000, "kw": ["pharma", "healthcar
     "why": "Boost high-value drug and formulation manufacturing and exports."},
    {"scheme": "Bulk drugs / APIs / key starting materials", "outlay_cr": 6940, "
     "why": "Reduce dependence on China for drug raw materials."},
    {"scheme": "Medical devices", "outlay_cr": 3420, "kw": ["medical", "healthcar
     "why": "Localise diagnostics, implants and equipment manufacturing."},
    {"scheme": "Telecom and networking products", "outlay_cr": 12195, "kw": ["tel
     "why": "Build domestic 5G, routers and network equipment champions."},
    {"scheme": "White goods (AC and LED)", "outlay_cr": 6238, "kw": ["consumer du
     "why": "Deepen local component supply for ACs and LED lighting."},
    {"scheme": "Food processing", "outlay_cr": 10900, "kw": ["food", "agro", "bev
     "why": "Create Indian food brands and boost processed-food exports."},
    {"scheme": "Textiles (man-made fibre and technical)", "outlay_cr": 10683, "kw
     "why": "Scale up MMF and technical textiles for export competitiveness."},
    {"scheme": "Specialty steel", "outlay_cr": 6322, "kw": ["steel", "metal"],
     "why": "Make high-grade steel locally instead of importing it."},
    {"scheme": "Drones and drone components", "outlay_cr": 120, "kw": ["aerospace
     "why": "Seed an Indian drone industry for civil and defence use."},
    {"scheme": "National Green Hydrogen Mission (incl. SIGHT)", "outlay_cr": 1974
     "why": "Electrolyser manufacturing and green hydrogen output to cut fossil i
]
PLI_QUERIES = ["PLI scheme beneficiaries approved companies crore", "PLI scheme i
               "PLI scheme approval company investment crore"]
BROKER_POS = ["buy", "upgrade", "outperform", "overweight", "accumulate", "initia
BROKER_NEG = ["sell", "downgrade", "underperform", "underweight", "reduce", "cuts

# ---- GOVERNMENT / POLICY TRACKING (edit freely) ----
# Google News searches per theme (last 30 days). Add or change phrases to follow
POLICY_QUERIES = {
    "Precision / Aerospace / Defence": ["Defence Acquisition Council approves", "
                                        "aerospace manufacturing India scheme"],
    "Transformers / T&D / Power equipment": ["transmission grid investment India
                                             "Cabinet approves power transmission
    "Specialty / Water chemicals": ["PLI specialty chemicals India", "Jal Jeevan
    "Semiconductor / Electronics": ["India Semiconductor Mission approval", "PLI
                                    "Cabinet approves electronics manufacturing I
    "Rail / Automation / Industrial": ["Railways tender Kavach order", "Cabinet a
}
POLICY_KW = {  # used to tag PIB / newsletter items to a theme
    "Precision / Aerospace / Defence": ["defence", "defense", "aerospace", "drdo"
    "Transformers / T&D / Power equipment": ["transmission", "grid", "transformer
    "Specialty / Water chemicals": ["chemical", "water", "jal jeevan", "anti-dump
```

```
    "Semiconductor / Electronics": ["semiconductor", "electronics", "chip", "pli"
    "Rail / Automation / Industrial": ["railway", "rail ", "kavach", "vande bhara
}
POLICY_POS = ["approve", "cabinet", "outlay", "scheme", "mission", "incentive", "
              "allocat", "budget", "subsidy", "indigenis", "boost", "anti-dumping
POLICY_NEG = ["scrap", "withdraw", "cancel", "delay", "curb", "ban on", "duty cut
PIB_FEED = "https://pib.gov.in/RssMain.aspx?ModId=6&Lang=1&Regid=3"   # official
NEWSLETTER_FEEDS = [
    # Paste RSS feed links of newsletters/blogs you follow, e.g. "https://yournam
]

# ---------------------------------------------------------------------------
# small helpers
# ---------------------------------------------------------------------------
# ===========================================================================
# v4: ALL-STOCK UNIVERSE, AREAS, CUSTOMERS, BROKERS  (these override older settin
# ===========================================================================
UNIVERSE_QUERY = "Market Capitalization > 5"   # Rs Cr. This pulls EVERY company
CANDIDATE_QUERY = UNIVERSE_QUERY
# Companies whose latest quarter is growing fast are loaded FIRST, so the best ca
PRIORITY_QUERY = ("Market Capitalization > 5 AND Net profit latest quarter > Net
                  "AND Sales latest quarter > Sales preceding year quarter")
MAX_PAGES = 300
MAX_CANDIDATES = 6000
BUDGET_MIN = 330            # time budget per full run (minutes). Unfinished work
FUND_REFRESH_DAYS = 6       # non-shortlisted stocks have their financials refres
ENRICH_PER_RUN = 250        # concall transcripts / credit reports read per run
FEATURED_MAX = 700          # stocks embedded in the dashboard page; every other
POLICY_EXTRA_MAX = 20
ONLY_THEMES = False

# Each AREA lists: tags (matched to Screener industry names / company name), text
# description and concalls), what drives it, what can go wrong, and policy news s
AREAS = {
    "Mining & minerals": {
        "tags": ["mining", "mineral", "coal"],
        "text": ["mining", "mineral exploration", "iron ore", "beneficiation", "r
        "drivers": "Infrastructure and steel demand, critical-mineral push, aucti
        "risks": "Commodity price swings, royalty and regulatory changes, environ
        "queries": ["Ministry of Mines critical minerals auction India", "Cabinet
    "Precision engineering & components": {
        "tags": ["precision", "forging", "castings", "bearing"],
        "text": ["precision engineering", "precision component", "precision machi
                 "close tolerance", "investment casting", "turned components", "m
        "drivers": "Global supply chains adding India as a source, long qualifica
        "risks": "Concentrated customers, lumpy orders, currency swings, premium
```

```
    "queries": ["precision engineering exports India aerospace supply chain",
"Electronics & EMS": {
    "tags": ["electronic"],
    "text": ["electronics manufacturing services", "pcb", "printed circuit",
             "mobile components", "enclosures", "camera module", "display mod
    "drivers": "PLI and component schemes, phone and consumer electronics loc
    "risks": "Thin margins in assembly, subsidy dependence, customer concentr
    "queries": ["PLI electronics components scheme approval", "Cabinet approv
"Semiconductors": {
    "tags": ["semiconductor"],
    "text": ["semiconductor", "osat", "chip design", "wafer", "fabless", "vls
             "ultra high purity", "test and assembly", "vacuum components"],
    "drivers": "India Semiconductor Mission incentives, fabs and OSAT plants
    "risks": "Long gestation, very high capital needs, technology and subsidy
    "queries": ["India Semiconductor Mission approval", "semiconductor fab OS
"Nuclear": {
    "tags": [],
    "name": ["nuclear"],
    "text": ["nuclear", "atomic energy", "npcil", "small modular reactor", "h
             "nuclear fuel", "fuel handling", "steam generator", "bhavini", "
    "drivers": "Government push to expand nuclear capacity, opening of the se
    "risks": "Very long project cycles, liability and approval uncertainty, s
    "queries": ["nuclear power India Cabinet approves", "small modular reacto
"Drones & unmanned systems": {
    "tags": [], "name": ["drone", "uav", "unmanned"],
    "text": ["drone", "uav", "unmanned aerial", "counter-drone", "loitering m
    "drivers": "Defence and civil drone demand, drone PLI, Make in India push
    "risks": "Early stage, order timing, fast-changing technology, regulatory
    "queries": ["India drone PLI scheme approval", "Ministry of Defence drone
"Green hydrogen & fuel cells": {
    "tags": [], "name": ["hydrogen", "electrolyser", "fuel cell"],
    "text": ["green hydrogen", "electrolyser", "electrolyzer", "fuel cell", "
    "drivers": "National Green Hydrogen Mission incentives, refinery and fert
    "risks": "Costs still high, policy dependent, long gestation periods.",
    "queries": ["National Green Hydrogen Mission tender India", "electrolyser
"Robotics & AI hardware": {
    "tags": [], "name": ["robot", "robotics"],
    "text": ["robot", "robotics", "machine vision", "cobot", "artificial inte
    "drivers": "Factory automation, rising labour costs, AI adoption and edge
    "risks": "Rich valuations, import dependence, execution risk.",
    "queries": ["India robotics manufacturing order", "AI hardware India manu
"Advanced materials & critical minerals": {
    "tags": [], "name": ["rare earth", "graphite", "titanium"],
    "text": ["rare earth", "magnets", "graphite", "titanium", "carbon fibre",
             "critical mineral", "tungsten", "cobalt"],
    "drivers": "National Critical Mineral Mission, EV and defence magnet dema
```

```
    "risks": "Commodity prices, long approvals, China's dominance of supply."
    "queries": ["National Critical Mineral Mission India", "rare earth magnet
"Telecom, 5G & fibre": {
    "tags": ["telecom equipment", "telecom - equipment"], "name": ["telecom",
    "text": ["5g", "fibre optic", "optical fibre", "optical fiber", "telecom
    "drivers": "5G rollout, telecom PLI, rising data demand, BharatNet fibre.
    "risks": "Operator capex cycles, Chinese competition, pricing pressure.",
    "queries": ["PLI telecom networking products India", "BharatNet fibre ord
"Medical devices & biotech": {
    "tags": ["medical equipment", "healthcare equipment", "hospital"], "name"
    "text": ["medical device", "medical devices", "cdmo", "biologics", "biosi
    "drivers": "Medical-device PLI, China+1 shift for CDMO work, rising healt
    "risks": "Regulatory approvals, price controls, long product cycles.",
    "queries": ["PLI medical devices India approval", "CDMO India order contr
"Ammunition & explosives": {
    "tags": ["explosives", "ammunition"],
    "name": ["ammunition", "explosive", "munition"],
    "text": ["ammunition", "artillery shell", "gun shell", "shells", "fuze",
             "small arms", "munitions"],
    "drivers": "Self-reliance in ammunition, replenishing stockpiles, rising
    "risks": "Lumpy government orders, safety incidents, working capital, exp
    "queries": ["Ministry of Defence ammunition order India", "artillery shel
"Transmission, transformers & T&D": {
    "tags": ["transformer", "electrical equipment", "cable", "power equipment
    "text": ["transformer", "switchgear", "transmission line", "power cable",
    "drivers": "Grid expansion, renewable evacuation, data-centre and industr
    "risks": "Copper and CRGO steel costs, new capacity from many players, va
    "queries": ["transmission grid investment India approved", "Ministry of P
"Defence": {
    "tags": ["defence", "defense"],
    "text": ["defence", "defense", "ammunition", "naval", "drdo", "armoured",
    "drivers": "Indigenisation lists, rising defence budgets, export push, lo
    "risks": "Lumpy orders, government payment cycles, premium valuations.",
    "queries": ["Defence Acquisition Council approves", "Ministry of Defence
"Aerospace": {
    "tags": ["aerospace"],
    "text": ["aerospace", "aero-engine", "aircraft component", "airframe", "t
    "drivers": "Global aircraft makers diversifying supply chains, engine-par
    "risks": "Long certification cycles, customer concentration, cyclicality
    "queries": ["aerospace manufacturing India scheme", "Airbus Boeing Safran
"Space": {
    "tags": [],
    "text": ["isro", "launch vehicle", "space segment", "spacetech", "satelli
    "drivers": "Opening of the space sector to private firms, ISRO launch cad
    "risks": "Small order sizes, execution delays, dependence on ISRO and gov
    "queries": ["India space sector policy private satellite ISRO", "IN-SPACe
```

```
"Data centers & cooling": {
    "tags": [],
    "text": ["data center", "data centre", "hyperscale", "liquid cooling", "p
    "drivers": "Cloud and AI demand, data-localisation, hyperscaler build-out
    "risks": "Lumpy orders, a few global customers, fast technology change.",
    "queries": ["India data centre investment policy", "data center cooling p
"Railways": {
    "tags": ["railway", "rail"],
    "text": ["indian railways", "railway wagons", "locomotive", "kavach", "me
    "drivers": "Rail capex, Kavach safety rollout, Vande Bharat and metro ord
    "risks": "Government order timing, working-capital stretch, tender compet
    "queries": ["Railways tender Kavach order", "Cabinet approves railway pro
"Specialty chemicals": {
    "tags": ["specialty chem", "chemical", "agrochem", "fluoro"],
    "text": ["specialty chemical", "fluorochemical", "agrochemical intermedia
    "drivers": "China+1 shift, regulatory approvals and sticky customers, imp
    "risks": "Chinese dumping, pricing cycles, destocking, environmental comp
    "queries": ["PLI specialty chemicals India", "anti-dumping duty chemicals
"Water & environment": {
    "tags": ["water"],
    "name": ["water"],
    "text": ["water treatment", "wastewater", "desalination", "water purifica
    "drivers": "Jal Jeevan Mission, industrial water reuse rules, urban water
    "risks": "Government payment delays, order timing, intense tender competi
    "queries": ["Jal Jeevan Mission water treatment tender", "wastewater recy
"Solar & renewables": {
    "tags": ["solar", "renewable", "wind energy", "wind turbine"],
    "text": ["solar module", "solar cell", "wind turbine", "inverter", "green
    "drivers": "Renewable capacity targets, solar PLI, green hydrogen mission
    "risks": "Module price cycles, Chinese competition, policy and tariff cha
    "queries": ["solar PLI scheme India modules", "green hydrogen mission Ind
"EV & batteries": {
    "tags": ["battery"],
    "text": ["electric vehicle", "lithium-ion", "battery pack", "ev charging"
    "drivers": "EV adoption incentives, ACC battery PLI, charging build-out,
    "risks": "Technology change, subsidy dependence, raw-material price swing
    "queries": ["ACC battery PLI India", "electric vehicle policy India subsi
"Industrial automation & capital goods": {
    "tags": ["industrial machinery", "automation", "compressors"],
    "text": ["industrial automation", "robotics", "motion control", "plc "],
    "drivers": "Manufacturing capex cycle, factory automation, import substit
    "risks": "Cyclical orders, working-capital strain, competition from impor
    "queries": ["industrial automation India scheme", "capital goods order in
"Oil, gas & process equipment": {
    "tags": ["oilfield", "oil equipment", "oil & gas equipment"],
    "text": ["oil and gas equipment", "heat exchanger", "pressure vessel", "p
```

```
        "drivers": "Refinery and petrochemical expansion, LNG and gas-grid build-
        "risks": "Capex cycles, oil-price swings, government customer concentrati
        "queries": ["refinery petrochemical expansion India approves", "city gas
    "Shipbuilding & marine": {
        "tags": ["shipbuilding"],
        "text": ["shipbuilding", "shipyard", "naval vessels", "offshore vessel"],
        "drivers": "Shipbuilding financial assistance, naval orders, coastal and
        "risks": "Long delivery cycles, working capital, dependence on government
        "queries": ["India shipbuilding policy financial assistance", "Cabinet ap
}
THEMES = {a: d["tags"] for a, d in AREAS.items()}
THEME_INFO = {a: {"drivers": d["drivers"], "risks": d["risks"]} for a, d in AREAS
POLICY_QUERIES = {a: d["queries"] for a, d in AREAS.items()}
POLICY_KW = {a: d["text"][:5] + d["tags"][:3] for a, d in AREAS.items()}

_CUST = {
    "Global": ["Safran", "Boeing", "Airbus", "GE Aerospace", "GE Vernova", "Gener
               "Honeywell", "Collins Aerospace", "RTX", "Raytheon", "Lockheed Mar
               "Embraer", "Bombardier", "Spirit AeroSystems", "Leonardo", "BAE Sy
               "Textron", "Bloom Energy", "Siemens Energy", "Siemens", "ABB", "Hi
               "Mitsubishi", "Toshiba", "Westinghouse", "Vestas", "Tesla", "Cummi
               "Schlumberger", "Halliburton", "Vertiv", "Nvidia", "Apple", "Samsu
               "Amazon", "AWS", "Google", "Intel", "Micron", "TSMC", "Qualcomm",
               "Wabtec", "Bosch", "Continental", "Hyundai", "Toyota", "Volkswagen
               "Rio Tinto", "BHP", "SpaceX", "Kongsberg", "Rheinmetall", "Saab",
               "Parker Hannifin", "Emerson", "Rockwell", "Johnson Controls", "Dai
               "Panasonic", "LG", "Xiaomi", "Danfoss", "Grundfos", "Xylem", "Pent
    "India": ["NPCIL", "NTPC", "BHEL", "Larsen & Toubro", "L&T", "Tata Power", "A
              "ONGC", "Coal India", "NMDC", "Vedanta", "Hindalco", "Tata Steel",
              "Hindustan Aeronautics", "BEL", "Bharat Electronics", "BDL", "Bhara
              "Indian Railways", "Indian Army", "Indian Navy", "Indian Air Force"
              "Mahindra", "Maruti", "Tata Electronics", "Bharat Forge", "Hindusta
              "HPCL", "Thermax", "Titagarh", "Jio", "Airtel", "BSNL", "Ministry o
}
_CUST["Global"] += ["ASML", "Applied Materials", "Lam Research", "KLA", "Tokyo El
                    "Pegatron", "Wistron", "Flex", "Jabil", "Oppo", "Vivo", "Moto
                    "Rosatom", "Holtec", "NuScale", "Framatome", "EDF", "AtkinsRe
                    "Howmet", "Precision Castparts", "Doosan", "Bechtel", "Fluor"
_CUST["India"] += ["Munitions India", "Armoured Vehicles Nigam", "AVNL", "Gun and
                   "Advanced Weapons and Equipment India", "BHAVINI", "Department
                   "Solar Industries", "Data Patterns", "Dixon", "GRSE", "Garden
CUSTOMERS = {n: reg for reg, names in _CUST.items() for n in names}
CUST_RE = re.compile(r"(?<![A-Za-z])(" + "|".join(re.escape(n) for n in sorted(CU
CUST_TRIG = re.compile(r"order|contract|supply|supplier|partner|agreement|mou|cus
FOCUS_RE = re.compile(r"focus(?:ing|ed)? on|we are targeting|targeting|entering|e
```

```
BROKERS = ["Kotak Institutional Equities", "Kotak", "ICICI Securities", "ICICI Di
           "Nuvama", "JM Financial", "HDFC Securities", "Jefferies", "Nomura", "G
           "UBS", "Macquarie", "Emkay", "Anand Rathi", "Sharekhan", "Prabhudas Li
           "Geojit", "SBI Securities", "Ambit", "Elara", "IIFL", "Investec", "Ber
           "Dolat", "Centrum", "Religare"]
BROKER_RE = re.compile(r"(?<![A-Za-z])(" + "|".join(re.escape(b) for b in sorted(
TARGET_RE = re.compile(r"target(?: price)?(?: of| at)?\s*(?:Rs\.?|INR|\u20b9)?\s*

def num(x):
    try:
        return float(re.sub(r"[,%\u20b9\s]", "", x))
    except Exception:
        return None

def mean(v):
    v = [x for x in v if x is not None]
    return sum(v) / len(v) if v else None

def med(v):
    v = [x for x in v if x is not None]
    return st.median(v) if v else None

def pct_true(v):
    return 100.0 * sum(1 for x in v if x) / len(v) if v else 0.0

def now_ist():
    return datetime.now(IST)

def get(url, **kw):
    for i in range(3):
        try:
            r = requests.get(url, headers=UA, timeout=25, **kw)
            if r.status_code == 429:
                time.sleep(3 * (i + 1)); continue
            return r
        except requests.RequestException:
            time.sleep(2)
    return None

# ---------------------------------------------------------------------------
# Screener.in
# ---------------------------------------------------------------------------
S = requests.Session()
S.headers.update(UA)

def login():
```

```
    r = S.get(f"{BASE}/login/")
    tok = BeautifulSoup(r.text, "html.parser").find("input", {"name": "csrfmiddle
    r = S.post(f"{BASE}/login/", data={
        "csrfmiddlewaretoken": tok,
        "username": os.environ["SCREENER_USER"],
        "password": os.environ["SCREENER_PASS"]}, headers={"Referer": f"{BASE}/lo
    if "/login" in r.url:
        sys.exit("Screener login failed - check SCREENER_USER / SCREENER_PASS sec

def _total_results(text):
    """Screener says how many companies match, e.g. '5,382 results found'. Return
    for pat in (r"([\d,]+)\s+results?\s+found", r"of\s+([\d,]+)\s+results?", r"([
        m = re.search(pat, text, re.I)
        if m:
            try:
                return int(m.group(1).replace(",", ""))
            except ValueError:
                pass
    return None

def screen_rows(query, max_pages=None):
    """All companies for one Screener query. Stops when a page brings nothing new
    rows, seen, total, page = [], set(), None, 1
    while page <= (max_pages or MAX_PAGES):
        try:
            r = S.get(f"{BASE}/screen/raw/", params={"query": query, "page": page
        except requests.RequestException:
            time.sleep(5)
            break
        soup = BeautifulSoup(r.text, "html.parser")
        if page == 1:
            total = _total_results(soup.get_text(" "))
        table = soup.select_one("table.data-table")
        if not table:
            break
        new = 0
        for tr in table.select("tr[data-row-company-id]"):
            a = tr.find("a", href=re.compile(r"^/company/"))
            if a and a["href"] not in seen:
                seen.add(a["href"])
                rows.append({"name": a.get_text(strip=True), "url": BASE + a["hre
                new += 1
        if new == 0 or len(rows) >= MAX_CANDIDATES:
            break
        page += 1
        time.sleep(0.8)
```

```
    return rows, total

def split_screen(lo=5.0, hi=1_000_000.0):
    """Screener shows only a limited number of rows per query, so list the market
    and split any band that is still cut short, until every band is complete."""
    out, seen, t0 = [], set(), time.time()
    def band(a, b, depth=0):
        rows, total = screen_rows(f"Market Capitalization > {a:.4f} AND Market Ca
        for r_ in rows:
            if r_["url"] not in seen:
                seen.add(r_["url"])
                out.append(r_)
        complete = (total is not None and len(rows) >= total * 0.99) or (total is
        if not complete and b / a > 1.02 and depth < 16 and time.time() - t0 < 40
            mid = (a * b) ** 0.5
            band(a, mid, depth + 1)
            band(mid, b, depth + 1)
    band(lo, hi)
    return out

def run_screen():
    rows, total = screen_rows(CANDIDATE_QUERY)
    print(f"Screener reports {total} matching companies; the single query returne
    if len(rows) >= 300 or (total and len(rows) >= total * 0.97):
        return rows
    print("One query is cut short by Screener - listing the whole market in marke
    rows = split_screen()
    print(f"Band listing found {len(rows)} companies")
    return rows

def series(soup, section, labels):
    sec = soup.find("section", id=section)
    if not sec:
        return [], ""
    head = sec.select("thead th")
    period = head[-1].get_text(strip=True) if head else ""
    for tr in sec.select("tbody tr"):
        td = tr.find("td")
        if td and any(td.get_text(strip=True).lower().startswith(l.lower()) for l
            return [num(x.get_text()) for x in tr.find_all("td")[1:]], period
    return [], period

def is_ath(v):
    c = [x for x in v if x is not None]
    return len(c) >= 4 and v[-1] is not None and v[-1] > 0 and v[-1] >= max(c)
```

```
def not_rising(v):
    c = [x for x in v if x is not None]
    if len(c) < 2:
        return None
    ok = c[-1] <= c[-2] * 1.03 + 0.5
    if len(c) >= 4:
        ok = ok and c[-1] <= mean(c[-4:-1]) * 1.05 + 0.5
    return ok

def yoy(v):
    if len(v) >= 5 and v[-1] is not None and v[-5] not in (None, 0) and v[-5] > 0
        return (v[-1] / v[-5] - 1) * 100
    return None

def theme_of(tags, name):
    hay = f"{tags} {name}".lower()
    for theme, kws in THEMES.items():
        if any(k in hay for k in kws):
            return theme
    return ""

def grab_table(soup, section, table_id=None):
    """Return (column headers, {normalised row label: values}) for a Screener tab
    sec = soup.find("section", id=section)
    if not sec:
        return [], {}
    scope = sec.find(id=table_id) if table_id else None
    scope = scope if scope is not None else sec
    tbl = scope if scope.name == "table" else scope.find("table")
    if tbl is None:
        return [], {}
    hd = [th.get_text(strip=True) for th in tbl.select("thead th")][1:]
    rows = {}
    for tr in tbl.select("tbody tr"):
        tds = tr.find_all("td")
        if len(tds) < 2:
            continue
        label = re.sub(r"[\s+]+$", "", tds[0].get_text().replace("\u00a0", " ").s
        rows.setdefault(label, [num(td.get_text()) for td in tds[1:]])
    return hd, rows

def results_view(hd, rows):
    """Is the latest quarter excellent or not? Compared YoY and QoQ."""
    qs = rows.get("sales") or rows.get("revenue") or []
    qp = rows.get("net profit") or []
    om = rows.get("opm %") or []
```

```
    n = len(qs)
    if n < 2 or len(qp) < 2 or qs[-1] is None:
        return None
    def g(a, b):
        return (a / b - 1) * 100 if a is not None and b is not None and b > 0 els
    r = {"period": hd[-1] if hd else "", "sales": qs[-1], "profit": qp[-1],
         "sales_yoy": g(qs[-1], qs[-5]) if n >= 5 else None, "sales_qoq": g(qs[-1
         "profit_yoy": g(qp[-1], qp[-5]) if len(qp) >= 5 else None, "profit_qoq":
         "opm": om[-1] if om else None, "opm_yoy": om[-5] if len(om) >= 5 else No
    pts = 0
    sy, py, sq, pq = r["sales_yoy"], r["profit_yoy"], r["sales_qoq"], r["profit_q
    if sy is not None:
        pts += 2 if sy >= 20 else 1 if sy >= 10 else -2 if sy < 0 else 0
    if py is not None:
        pts += 2 if py >= 25 else 1 if py >= 10 else -2 if py < 0 else 0
    if sq is not None:
        pts += 1 if sq > 0 else -1
    if pq is not None:
        pts += 1 if pq > 0 else -1
    if r["opm"] is not None and r["opm_yoy"] is not None:
        pts += 1 if r["opm"] > r["opm_yoy"] else -1
    if qp[-1] is not None and qp[-1] < 0:
        verdict = "Loss-making"
    else:
        verdict = "Excellent" if pts >= 5 else "Good" if pts >= 3 else "Mixed" if
    trend = [g(qp[i], qp[i - 4]) for i in range(max(4, len(qp) - 4), len(qp))]
    tt = [x for x in trend if x is not None]
    r.update({"verdict": verdict, "pts": pts, "yoy_trend": [round(x) for x in tt]
              "accelerating": len(tt) >= 3 and tt[-1] > tt[-2] > tt[-3],
              "hist": [{"q": hd[i] if i < len(hd) else "", "s": qs[i], "p": qp[i]
                        for i in range(max(0, n - 6), n)]})
    return r

def holding_view(hd, rows):
    """Who owns the company, and are institutions coming in or leaving?"""
    keys = ["promoters", "fiis", "diis", "government", "public"]
    ser = {k: rows.get(k) or [] for k in keys}
    if not ser["promoters"] and not ser["fiis"]:
        return None
    holders = rows.get("no. of shareholders") or []
    def chg(v, k):
        return v[-1] - v[-1 - k] if len(v) > k and v[-1] is not None and v[-1 - k
    ch = {k: {"now": ser[k][-1] if ser[k] else None, "q": chg(ser[k], 1), "y": ch
    sig = []
    fy, dy, py = ch["fiis"]["y"], ch["diis"]["y"], ch["promoters"]["y"]
    if fy is not None and fy >= 1:
```

```
        sig.append({"t": f"FIIs (foreign funds) raised holding by {fy:.1f} pts in
    elif fy is not None and fy <= -1:
        sig.append({"t": f"FIIs (foreign funds) cut holding by {abs(fy):.1f} pts
    if dy is not None and dy >= 1:
        sig.append({"t": f"Domestic institutions (mutual funds, insurers) raised
    elif dy is not None and dy <= -1:
        sig.append({"t": f"Domestic institutions (mutual funds, insurers) cut hol
    for k, lab in (("fiis", "FII"), ("diis", "mutual fund / DII")):
        v = ser[k]
        if len(v) >= 5 and v[-5] is not None and v[-1] is not None and v[-5] < 1
            sig.append({"t": f"New {lab} entry: {v[-5]:.1f}% to {v[-1]:.1f}% in a
    if py is not None and py <= -2:
        sig.append({"t": f"Promoter holding fell {abs(py):.1f} pts in a year", "t
    elif py is not None and py >= 1:
        sig.append({"t": f"Promoters raised their stake by {py:.1f} pts in a year
    if len(holders) >= 5 and holders[-1] and holders[-5] and holders[-5] > 0:
        up = (holders[-1] / holders[-5] - 1) * 100
        if up >= 25:
            sig.append({"t": f"Shareholder count up {up:.0f}% in a year - retail
    inst = (fy or 0) + (dy or 0)
    state = "Accumulating" if inst >= 1.5 else "Reducing" if inst <= -1.5 else "S
    return {"labels": hd[-8:], "rows": {k: v[-8:] for k, v in ser.items() if v},
            "ch": ch, "signals": sig, "inst": state}

def insider_news(name):
    """Promoter / director / insider share purchases or sales (news about SEBI in
    nm = clean_name(name)
    first = nm.split()[0].lower() if nm.split() else ""
    r = get("https://news.google.com/rss/search", params={
        "q": f'"{nm}" promoter OR director OR insider OR chairman "buys shares" O
             f'"raises stake" OR "acquires shares" OR "sells shares" OR offloads
        "hl": "en-IN", "gl": "IN", "ceid": "IN:en"})
    out = []
    if r is None:
        return out
    buy = ("buys", "bought", "acquires", "picks up", "increases stake", "raises s
    sell = ("sells", "sold", "offloads", "pares stake", "reduces stake", "cuts st
    who = ("promoter", "director", "insider", "chairman", "managing director", "c
    for it in feed_items(r.content, 10):
        low = it["title"].lower()
        if (len(first) >= 3 and first not in low) or not any(w in low for w in wh
            continue
        b, s_ = any(w in low for w in buy), any(w in low for w in sell)
        if b and not s_:
            it["kind"] = "Insider buying"
        elif s_ and not b:
```

```
            it["kind"] = "Insider selling"
        else:
            continue
        out.append(it)
    out.sort(key=lambda i: i["age_h"])
    return out[:6]

def investor_news(name):
    """Bulk/block deals, fund and FII stake news, preferential issues, QIPs, warr
    nm = clean_name(name)
    first = nm.split()[0].lower() if nm.split() else ""
    out, seen = [], set()
    queries = [(f'"{nm}" bulk deal OR block deal OR mutual fund OR FII OR stake',
                (f'"{nm}" preferential issue OR preferential allotment OR warrants
    for q, win in queries:
        r = get("https://news.google.com/rss/search", params={"q": f"{q} when:{wi
        if r is None:
            continue
        for it in feed_items(r.content, 8):
            low, key = it["title"].lower(), it["title"].lower()[:70]
            if key in seen or (len(first) >= 3 and first not in low):
                continue
            if any(k in low for k in ("preferential", "warrant", "qip", "rights i
                it["kind"] = "Preferential / fund raise"
            elif any(k in low for k in ("bulk deal", "block deal", "stake", "mutu
                it["kind"] = "Big investor"
            else:
                continue
            seen.add(key); out.append(it)
        time.sleep(0.4)
    out.sort(key=lambda i: i["age_h"])
    return out[:8]

def margin_view(rows):
    """Cost quality: costs (COGS proxy) should fall as a share of sales, margin s
    qs = rows.get("sales") or rows.get("revenue") or []
    ex = rows.get("expenses") or []
    op = rows.get("operating profit") or []
    om = rows.get("opm %") or []
    if len(qs) < 5 or len(ex) < 5 or not qs[-1]:
        return None
    def cr(i):
        return ex[i] / qs[i] * 100 if qs[i] and ex[i] is not None and qs[i] > 0 e
    def g(a, b):
        return (a / b - 1) * 100 if a is not None and b is not None and b > 0 els
    c_now, c_yr, c_q = cr(-1), cr(-5), cr(-2)
```

```
    o_now = om[-1] if om else None
    o_yr = om[-5] if len(om) >= 5 else None
    o_q = om[-2] if len(om) >= 2 else None
    op_yoy = g(op[-1], op[-5]) if len(op) >= 5 else None
    op_qoq = g(op[-1], op[-2]) if len(op) >= 2 else None
    sy = g(qs[-1], qs[-5])
    cost_fall = c_now is not None and c_yr is not None and c_now < c_yr - 0.2
    opm_up = o_now is not None and o_yr is not None and o_now > o_yr + 0.2
    opm_up_q = o_now is not None and o_q is not None and o_now >= o_q
    op_grow = op_yoy is not None and op_yoy >= 10
    op_lev = op_yoy is not None and sy is not None and op_yoy >= sy
    op_rec = bool(op) and is_ath(op)
    pts = sum([cost_fall, opm_up, op_grow, op_lev, opm_up_q, op_rec])
    weak = (o_now is not None and o_yr is not None and o_now < o_yr - 0.5) or (op
    verdict = "Excellent" if (cost_fall and opm_up and op_grow) else "Good" if ((
    return {"verdict": verdict, "cost_now": c_now, "cost_yr": c_yr, "cost_q": c_q
            "op_yoy": op_yoy, "op_qoq": op_qoq, "op_record": op_rec, "cost_fall":

def fetch_pm(html, url):
    """Plant & machinery growth from Screener's fixed-asset schedule. Best effort
    try:
        info = BeautifulSoup(html, "html.parser").find(id="company-info")
        wid = (info.get("data-warehouse-id") or info.get("data-company-id")) if i
        if not wid:
            return {}
        params = {"parent": "Fixed Assets", "section": "balance-sheet"}
        if "/consolidated/" in url:
            params["consolidated"] = ""
        r = S.get(f"{BASE}/api/company/{wid}/schedules/", params=params, timeout=
        if r.status_code != 200:
            return {}
        for k, v in r.json().items():
            if "plant" in k.lower() and isinstance(v, dict):
                vals = [num(str(x)) for x in v.values()]
                if len(vals) >= 2 and vals[-1] is not None and vals[-2] and vals[
                    return {"pm_now": vals[-1], "pm_prev": vals[-2], "pm_yoy": (v
    except Exception:
        pass
    return {}

def fetch_cogs(html, url):
    """Material cost % (true COGS) from Screener's schedule data. Best effort: re
    try:
        info = BeautifulSoup(html, "html.parser").find(id="company-info")
        wid = (info.get("data-warehouse-id") or info.get("data-company-id")) if i
        if not wid:
```

```
            return None
        params = {"parent": "Expenses", "section": "quarters"}
        if "/consolidated/" in url:
            params["consolidated"] = ""
        r = S.get(f"{BASE}/api/company/{wid}/schedules/", params=params, timeout=
        if r.status_code != 200:
            return None
        for k, v in r.json().items():
            if k.lower().startswith("material cost") and isinstance(v, dict):
                vals = [num(str(x)) for x in v.values()]
                if len(vals) >= 5 and vals[-1] is not None and vals[-5] is not No
                    return {"now": vals[-1], "yr": vals[-5], "q": vals[-2]}
    except Exception:
        pass
    return None

def parse_company(html, row):
    soup = BeautifulSoup(html, "html.parser")
    ratios = {}
    for li in soup.select("#top-ratios li"):
        n = li.select_one(".name")
        vs = [num(v.get_text()) for v in li.select(".number")]
        if n and vs:
            ratios[n.get_text(strip=True)] = vs
    tag_list = [a.get_text(strip=True) for a in soup.select("#peers a[href^='/mar
    tags = " | ".join(tag_list)
    sector = tag_list[0] if tag_list else "Other"
    qs, period = series(soup, "quarters", ["Sales", "Revenue"])
    qp, _ = series(soup, "quarters", ["Net Profit"])
    ys, _ = series(soup, "profit-loss", ["Sales", "Revenue"])
    yp, _ = series(soup, "profit-loss", ["Net Profit"])
    opm, _ = series(soup, "quarters", ["OPM"])
    pyoy, syoy = yoy(qp), yoy(qs)
    prior4 = [x for x in qp[-5:-1] if x is not None]
    breakout = bool(pyoy is not None and pyoy >= BREAKOUT_PROFIT_YOY and qp[-1] i
                    and prior4 and qp[-1] > max(prior4))
    f = {
        "period": period,
        "q_sales": qs[-1] if qs else None, "q_profit": qp[-1] if qp else None,
        "sales_yoy": syoy, "profit_yoy": pyoy,
        "opm_now": opm[-1] if opm else None, "opm_yr": opm[-5] if len(opm) >= 5 e
        "roce": (ratios.get("ROCE") or [None])[0], "pe": (ratios.get("Stock P/E")
        "mcap": (ratios.get("Market Cap") or [None])[0],
        "ath_q_sales": is_ath(qs), "ath_q_profit": is_ath(qp),
        "ath_y_sales": is_ath(ys), "ath_y_profit": is_ath(yp),
        "profit_breakout": breakout, "y_sales": ys[-1] if ys else None,
```

```
}
dd, _ = series(soup, "ratios", ["Debtor Days"])
iv, _ = series(soup, "ratios", ["Inventory Days"])
oks = [x for x in (not_rising(dd), not_rising(iv)) if x is not None]
f.update({"debtor_days": dd[-1] if dd else None, "debtor_prev": dd[-2] if len
          "inv_days": iv[-1] if iv else None, "inv_prev": iv[-2] if len(iv) >
          "wc_ok": (all(oks) if oks else None)})
def cagr_(v, n):
    if len(v) > n and v[-1] is not None and v[-1 - n] is not None and v[-1 -
        return ((v[-1] / v[-1 - n]) ** (1.0 / n) - 1) * 100
    return None
f["cagr"] = {"sales3": cagr_(ys, 3), "sales5": cagr_(ys, 5), "profit3": cagr_
try:
    hdp, rowp = grab_table(soup, "profit-loss")
    npv = list(rowp.get("net profit") or [])
    if hdp and hdp[-1].strip().upper() == "TTM":
        npv = npv[:-1]
    cfo = list((grab_table(soup, "cash-flow")[1]).get("cash from operating ac
    n3 = min(3, len(npv), len(cfo))
    if n3 >= 2 and None not in (npv[-n3:] + cfo[-n3:]) and sum(npv[-n3:]) > 0
        f["cfo_ratio"] = sum(cfo[-n3:]) / sum(npv[-n3:])
    bs = grab_table(soup, "balance-sheet")[1]
    br_, eq_, rv_ = bs.get("borrowings") or [], bs.get("equity capital") or [
    if br_ and eq_ and rv_ and None not in (br_[-1], eq_[-1], rv_[-1]) and (e
        f["de"] = br_[-1] / (eq_[-1] + rv_[-1])
except Exception:
    pass
try:
    bs2 = grab_table(soup, "balance-sheet")[1]
    cf2 = grab_table(soup, "cash-flow")[1]
    fa, cw = bs2.get("fixed assets") or [], bs2.get("cwip") or []
    cfo2, cfi2 = cf2.get("cash from operating activity") or [], cf2.get("cash
    cx = {}
    if len(fa) >= 2 and fa[-1] is not None and fa[-2] and fa[-2] > 0:
        cx["fa_yoy"] = (fa[-1] / fa[-2] - 1) * 100
    if len(fa) >= 4 and fa[-1] and fa[-4] and fa[-4] > 0 and fa[-1] > 0:
        cx["fa_cagr3"] = ((fa[-1] / fa[-4]) ** (1 / 3) - 1) * 100
    if cw and cw[-1] is not None and fa and fa[-1]:
        cx["cwip"] = cw[-1]
        cx["cwip_pct"] = cw[-1] / fa[-1] * 100
    if len(cfo2) >= 2 and None not in (cfo2[-1], cfo2[-2]):
        cx["cfo_now"], cx["cfo_prev"] = cfo2[-1], cfo2[-2]
        if cfo2[-2] > 0:
            cx["cfo_yoy"] = (cfo2[-1] / cfo2[-2] - 1) * 100
    if cfi2 and cfi2[-1] is not None:
        cx["cfi_now"] = cfi2[-1]
```

```
    f["capex"] = cx
except Exception:
    f["capex"] = {}
try:
    hdp2, rowp2 = grab_table(soup, "profit-loss")
    sal2 = list(rowp2.get("sales") or rowp2.get("revenue") or [])
    npr2, hd2, ttm_p = list(rowp2.get("net profit") or []), list(hdp2), None
    if hd2 and hd2[-1].strip().upper() == "TTM":
        ttm_p = npr2[-1] if npr2 else None
        sal2, npr2, hd2 = sal2[:-1], npr2[:-1], hd2[:-1]
    if len(npr2) >= 2 and len(sal2) >= 2:
        def g2_(a, b):
            return (a / b - 1) * 100 if a is not None and b is not None and b
        f["annual"] = {"fy": hd2[-1] if hd2 else "", "sales": sal2[-1], "prof
                       "prev_profit": npr2[-2], "profit_yoy": g2_(npr2[-1], n
                       "ttm_profit": ttm_p,
                       "hist": [{"y": hd2[i] if i < len(hd2) else "", "s": sa
except Exception:
    pass
qh, qrows = grab_table(soup, "quarters")
f["results"] = results_view(qh, qrows)
f["margin"] = margin_view(qrows)
f["pledged"] = (ratios.get("Pledged percentage") or [None])[0]
sh, srows = grab_table(soup, "shareholding", "quarterly-shp")
holding = holding_view(sh, srows)
docs = {"transcript": "", "ppt": "", "ratings": [], "filings": []}
dsec = soup.find("section", id="documents")
if dsec:
    for a in dsec.find_all("a", href=True):
        href = a["href"]
        href = BASE + href if href.startswith("/") else href
        if not href.startswith("https://"):
            continue
        txt, low = a.get_text(" ", strip=True), a.get_text(" ", strip=True).l
        if low == "transcript" and not docs["transcript"]:
            docs["transcript"] = href
        elif low == "ppt" and not docs["ppt"]:
            docs["ppt"] = href
        elif "rating" in low and len(docs["ratings"]) < 2:
            docs["ratings"].append({"text": txt, "href": href})
        elif any(k in low for k in FILING_KW) and len(docs["filings"]) < 8:
            docs["filings"].append({"text": txt[:140], "href": href})
m = re.search(r"/company/([^/]+)/", row["url"])
sym = m.group(1) if m else row["name"]
h1 = soup.find("h1")
nm = h1.get_text(strip=True) if ((row.get("watch") or row.get("guess")) and h
```

```
    ab = soup.select_one(".about") or soup.select_one(".company-profile")
    about = ab.get_text(" ", strip=True)[:1500] if ab else ""
    areas = areas_of(tags, nm, about)
    return {"sym": sym, "name": nm, "url": row["url"], "tags": tags, "sector": se
            "theme": areas[0] if areas else sector, "priority": bool(areas), "doc
            "holding": holding, "watch": bool(row.get("watch"))}

# ---------------------------------------------------------------------------
# Yahoo Finance: price / volume
# ---------------------------------------------------------------------------
def fetch_chart(ysym, rng="1y"):
    r = get(f"https://query1.finance.yahoo.com/v8/finance/chart/{requests.utils.q
            params={"range": rng, "interval": "1d"})
    try:
        res = r.json()["chart"]["result"][0]
        q = res["indicators"]["quote"][0]
        rows = [(c, h, lo, v or 0) for c, h, lo, v in zip(q["close"], q["high"],
                if c is not None and h is not None and lo is not None]
        return {"close": [x[0] for x in rows], "high": [x[1] for x in rows], "low
    except Exception:
        return None

def adx_calc(h, lo, c, n=14):
    """Wilder's Average Directional Index. Above 30 = very strong trend. Returns
    if len(c) < 2 * n + 5:
        return None, None, None
    tr, pdm, mdm = [], [], []
    for i in range(1, len(c)):
        up, dn = h[i] - h[i - 1], lo[i - 1] - lo[i]
        pdm.append(up if up > dn and up > 0 else 0.0)
        mdm.append(dn if dn > up and dn > 0 else 0.0)
        tr.append(max(h[i] - lo[i], abs(h[i] - c[i - 1]), abs(lo[i] - c[i - 1])))
    def sm(x):
        s_ = sum(x[:n]); out = [s_]
        for v_ in x[n:]:
            s_ = s_ - s_ / n + v_
            out.append(s_)
        return out
    a_s, p_s, m_s = sm(tr), sm(pdm), sm(mdm)
    dx, pdi, mdi = [], None, None
    for a_, p_, m_ in zip(a_s, p_s, m_s):
        if a_ == 0:
            dx.append(0.0); continue
        pdi, mdi = 100 * p_ / a_, 100 * m_ / a_
        dx.append(100 * abs(pdi - mdi) / (pdi + mdi) if pdi + mdi else 0.0)
    if len(dx) < n:
```

```
        return None, None, None
    adx = sum(dx[:n]) / n
    for x_ in dx[n:]:
        adx = (adx * (n - 1) + x_) / n
    return round(adx, 1), round(pdi, 1), round(mdi, 1)

def atr_calc(h, lo, c, n=14):
    tr = [max(h[i] - lo[i], abs(h[i] - c[i - 1]), abs(lo[i] - c[i - 1])) for i in
    return mean(tr[-n:]) if len(tr) >= n else None

_NIFTY = {}
def nifty_returns():
    """Nifty 50 returns, used to judge relative strength. Fetched once per run.""
    if not _NIFTY:
        d = fetch_chart("^NSEI", "2y")
        if d and len(d["close"]) > 130:
            c = d["close"]
            _NIFTY.update(r3=(c[-1] / c[-64] - 1) * 100, r6=(c[-1] / c[-127] - 1)
        else:
            _NIFTY["none"] = True
    return _NIFTY

def market_metrics(sym):
    d = None
    for suf in (".NS", ".BO"):
        d = fetch_chart(sym + suf, "2y")
        if d and len(d["close"]) >= 60:
            break
    else:
        return None
    c, h, v, lo_ = d["close"], d["high"], d["vol"], d["low"]
    price, prev = c[-1], c[-2]
    avg20 = mean(v[-21:-1])

    def sma(n, back=0):
        end = len(c) - back
        return mean(c[end - n:end]) if end - n >= 0 else None
    gap_now = state = cross_days = None
    s50, s200 = sma(50), sma(200)
    if s50 and s200:
        gap_now = (s50 / s200 - 1) * 100
        cur = s50 >= s200
        for back in range(1, 25):  # look back ~5 weeks for a crossover
            a, b = sma(50, back), sma(200, back)
            if a is None or b is None:
                break
```

```
            if (a >= b) != cur:
                cross_days = back
                break
        slope = s50 - (sma(50, 5) or s50)
        if cross_days is not None and cur:
            state = "golden_cross"
        elif cross_days is not None and not cur:
            state = "death_cross"
        elif abs(gap_now) <= 3 and slope > 0 and price > s50:
            state = "converging_up"
        elif abs(gap_now) <= 3:
            state = "squeeze"
    adx_v, pdi_v, mdi_v = adx_calc(h, lo_, c)
    def rt_(k):
        return (price / c[-1 - k] - 1) * 100 if len(c) > k else None
    rs_raw = None
    if len(c) > 250:
        rs_raw = 0.4 * rt_(63) + 0.2 * rt_(126) + 0.2 * rt_(189) + 0.2 * rt_(250)
    nf = nifty_returns()
    r6 = rt_(126)
    base20 = mean(v[-23:-3])
    return {
        "adx": adx_v, "pdi": pdi_v, "mdi": mdi_v, "atr": atr_calc(h, lo_, c), "rs
        "rs_nifty6m": (r6 - nf["r6"]) if (r6 is not None and nf.get("r6") is not
        "ret2d": rt_(2), "ret5d": rt_(5), "ret6m": r6,
        "vol_max3": (max(v[-3:]) / base20) if base20 else None,
        "price": price, "day": (price / prev - 1) * 100,
        "dma50": mean(c[-50:]), "dma200": mean(c[-200:]) if len(c) >= 200 else No
        "hi52": max(h[-252:]),
        "vol": v[-1], "avg20": avg20, "vol_ratio": (v[-1] / avg20) if avg20 else
        "turnover_cr": (price * avg20 / 1e7) if avg20 else None,
        "ret1m": (price / c[-22] - 1) * 100 if len(c) > 22 else None,
        "ret3m": (price / c[-64] - 1) * 100 if len(c) > 64 else None,
        "asof": now_ist().strftime("%d %b %H:%M"),
        "dma_gap": gap_now, "dma_state": state, "cross_days": cross_days,
    }

def market_overview():
    out = {}
    for key, sym in (("nifty", "^NSEI"), ("vix", "^INDIAVIX")):
        d = fetch_chart(sym)
        if d and len(d["close"]) > 60:
            c = d["close"]
            out[key] = {"price": c[-1], "dma50": mean(c[-50:]),
                        "dma200": mean(c[-200:]) if len(c) >= 200 else None,
                        "ret1m": (c[-1] / c[-22] - 1) * 100}
```

```
    n, vx = out.get("nifty"), out.get("vix")
    if n:
        weak = n["dma200"] and n["price"] < n["dma200"]
        vol = vx and vx["price"] > 20
        if weak:
            out["mood"] = "Weak - Nifty below 200-DMA. Keep positions small, favo
            out["tone"] = "r"
        elif vol:
            out["mood"] = "Volatile - India VIX above 20. Stagger entries, expect
            out["tone"] = "a"
        elif n["ret1m"] > 0:
            out["mood"] = "Constructive - Nifty above 200-DMA and rising. Breakou
            out["tone"] = "g"
        else:
            out["mood"] = "Sideways - uptrend intact but momentum paused. Be sele
            out["tone"] = "a"
    return out

# ---------------------------------------------------------------------------
# News (Google News RSS) with simple sentiment
# ---------------------------------------------------------------------------
def clean_name(n):
    return re.sub(r"\b(Ltd|Limited|Pvt|Private|India)\.?\b", "", n, flags=re.I).s

def get_news(name, max_items=5, extra="stock", window="7d"):
    r = get("https://news.google.com/rss/search",
            params={"q": f'"{clean_name(name)}" {extra} when:{window}', "hl": "en
    items = []
    try:
        root = ET.fromstring(r.content)
        for it in root.iter("item"):
            title = (it.findtext("title") or "").strip()
            link = (it.findtext("link") or "").strip()
            if not link.startswith("http"):
                continue
            dt = parsedate_to_datetime(it.findtext("pubDate"))
            age_h = (datetime.now(timezone.utc) - dt).total_seconds() / 3600
            low = title.lower()
            pos = sum(w in low for w in POS_WORDS)
            neg = sum(w in low for w in NEG_WORDS)
            ago = f"{int(age_h)}h ago" if age_h < 48 else f"{int(age_h/24)}d ago"
            items.append({"title": title, "link": link, "src": (it.findtext("sour
                          "ago": ago, "age_h": round(age_h, 1), "s": (pos > neg)
            if len(items) >= max_items:
                break
    except Exception:
```

```
        pass
    return items

# ---------------------------------------------------------------------------
# Concall / order book / credit rating / brokerage / PLI per stock
# ---------------------------------------------------------------------------
AMT_RE = re.compile(r"(?:Rs\.?|INR|\u20b9)\s?([\d,]+(?:\.\d+)?)\s?(lakh crore|cro
VAL_RE = re.compile(r"(?:Rs\.?|INR|\u20b9)\s?([\d,]+(?:\.\d+)?)\s?(lakh crore|cro
BOOK_RE = re.compile(r"order\s*(book|backlog)", re.I)
FLOW_RE = re.compile(r"order\s*(inflow|pipeline|intake)", re.I)
RATING_RE = re.compile(r"\b(AAA|AA\+|AA-|AA|A\+|A-|BBB\+|BBB-|BBB|BB\+|BB-|BB)(?!
POS_T = ["strong", "robust", "confident", "record", "healthy", "visibility", "mom
NEG_T = ["headwind", "challeng", "weak", "delay", "pressure", "slowdown", "uncert

def find_amt(text):
    m = AMT_RE.search(text)
    return m.group(0).strip() if m and m.group(2) else ""

def to_cr(num_s, unit):
    try:
        v, u = float(num_s.replace(",", "")), unit.lower()
    except Exception:
        return None
    if u.startswith("lakh"):
        return v * 1e5
    if u.startswith("cr"):
        return v
    if u in ("bn", "billion"):
        return v * 100
    if u in ("mn", "million"):
        return v / 10
    return None

def pdf_text(url, max_pages=40):
    try:
        r = requests.get(url, headers={**UA, "Referer": "https://www.screener.in/
        if r.status_code != 200 or len(r.content) > 15_000_000:
            return ""
        if r.content[:4] == b"%PDF":
            from pypdf import PdfReader
            rd = PdfReader(io.BytesIO(r.content))
            return " ".join((pg.extract_text() or "") for pg in rd.pages[:max_pag
        return BeautifulSoup(r.text, "html.parser").get_text(" ")
    except Exception:
        return ""
```

```
CERT_KW = ["AS9100", "AS 9100", "NADCAP", "ASME", "N-stamp", "NQA-1", "IATF 16949
QUAL_RE = re.compile(r"approved vendor|vendor approval|qualif(?:ied|ication)|firs
                     r"long[- ]term agreement|\bLTA\b|framework agreement|rate co
TOPCUST_RE = re.compile(r"(?:top|largest|biggest|single|one)\s+(?:\w+\s+){0,2}cus

def analyse_transcript(text):
    text = re.sub(r"\s+", " ", text)
    sents = [x.strip() for x in re.split(r"(?<=[.?!])\s+", text) if 40 <= len(x)
    cut = lambda x: x[:220]
    book = [x for x in sents if BOOK_RE.search(x) and re.search(r"\d", x)]
    flow = [x for x in sents if FLOW_RE.search(x) and re.search(r"\d", x)]
    ob_cr, ob_text = None, ""
    for x in book:
        m = VAL_RE.search(x)
        if m:
            v = to_cr(m.group(1), m.group(2))
            if v:
                ob_cr, ob_text = v, f"Order book about Rs {v:,.0f} Cr"
                break
    guid = [x for x in sents if re.search(r"guidance|we expect|outlook|aim to|con
    capex = [x for x in sents if re.search(r"capex|capacity|expansion|commission"
    risks = [x for x in sents if re.search(r"headwind|challeng|delay|pressure|slo
    low = text.lower()
    pos, neg = sum(low.count(w) for w in POS_T), sum(low.count(w) for w in NEG_T)
    ratio = pos / (neg + 1)
    tone = "Positive" if ratio >= 3 else "Cautious" if ratio <= 1.2 else "Mixed"
    return {"tone": tone, "ob_cr": ob_cr, "ob_text": ob_text, "order_book": [cut(
            "guidance": [cut(x) for x in guid[:2]], "capex": [cut(x) for x in cap
            "targets": target_areas(low), "focus": [cut(x) for x in sents if FOCU
            "customers": customer_scan(sents),
            "certs": sorted({c for c in CERT_KW if re.search(r"(?<![A-Za-z0-9])"
            "quals": [cut(x) for x in sents if QUAL_RE.search(x)][:3],
            "top_cust_pct": next((int(m_.group(1)) for m_ in (TOPCUST_RE.search(x

def parse_rating(text, link_text):
    m = RATING_RE.search(text or "")
    ag = re.search(r"from\s+([\w .&-]+)$", link_text.strip())
    dt = re.search(r"(\d{1,2}\s+\w{3}\s+\d{4})", link_text)
    return {"rating": f"{m.group(1)}/{m.group(2)}" if m else "", "agency": ag.gro
            "date": dt.group(1) if dt else ""}

def enrich(s, prev):
    """Deep check: latest concall transcript + latest credit rating report. Cache
    d = s.get("docs") or {}
    tr = d.get("transcript")
    if tr:
```

```
        if (prev.get("concall") or {}).get("link") == tr and (prev.get("concall")
            s["concall"] = prev["concall"]
        else:
            txt = pdf_text(tr)
            if txt:
                c = analyse_transcript(txt)
                c["link"], c["ppt"] = tr, d.get("ppt", "")
                s["concall"] = c
            time.sleep(1)
    rl = (d.get("ratings") or [None])[0]
    if rl:
        if (prev.get("credit") or {}).get("link") == rl["href"]:
            s["credit"] = prev["credit"]
        else:
            c = parse_rating(pdf_text(rl["href"], max_pages=4), rl["text"])
            c["link"] = rl["href"]
            s["credit"] = c
            time.sleep(1)

def pli_stock(name):
    out = []
    for it in get_news(name, max_items=8, extra="PLI scheme incentive"):
        low = it["title"].lower()
        if "pli" in low or "production linked" in low or "production-linked" in l
            it["amt"] = find_amt(it["title"])
            out.append(it)
    return out[:3]

def areas_of(tags, name, about):
    tg, nm, ab = tags.lower(), name.lower(), " " + (about or "").lower() + " "
    out = []
    for a, d in AREAS.items():
        if any(k in tg or k in nm for k in d["tags"] + d.get("name", [])) or any(
            out.append(a)
    return out

def target_areas(low):
    low = " " + low + " "
    sc = {a: sum(low.count(k) for k in d["text"]) for a, d in AREAS.items()}
    top = sorted(((n, a) for a, n in sc.items() if n >= 3), reverse=True)[:4]
    return [{"area": a, "n": n} for n, a in top]

def customer_scan(sents, src="concall", link=""):
    found = {}
    for x in sents:
        if not CUST_TRIG.search(x):
```

```
            continue
        for nm_ in set(CUST_RE.findall(x)):
            d = found.setdefault(nm_, {"name": nm_, "region": CUSTOMERS[nm_], "n"
            d["n"] += 1
            if not d["ctx"]:
                d["ctx"] = x[:220]
    return sorted(found.values(), key=lambda d: -d["n"])[:8]

def amt_cr_of(text):
    m = VAL_RE.search(text or "")
    return to_cr(m.group(1), m.group(2)) if m else None

def orders_news(name):
    nm = clean_name(name)
    first = nm.split()[0].lower() if nm.split() else ""
    r = get("https://news.google.com/rss/search", params={
        "q": f'"{nm}" order OR contract OR partnership OR "supply agreement" OR M
        "hl": "en-IN", "gl": "IN", "ceid": "IN:en"})
    out = []
    if r is None:
        return out
    for it in feed_items(r.content, 14):
        low = it["title"].lower()
        if len(first) >= 3 and first not in low:
            continue
        if not re.search(r"order|contract|partner|supply|agreement|mou|tie-up|col
            continue
        it["cust"] = [d["name"] for d in customer_scan([it["title"]], "news", it[
        it["amt_cr"] = amt_cr_of(it["title"])
        out.append(it)
    return out[:8]

def broker_view(name):
    out, seen = [], set()
    for extra in ("target price brokerage rating", "Kotak OR ICICI OR Motilal OR
        for it in get_news(name, max_items=8, extra=extra, window="60d"):
            key = it["title"].lower()[:70]
            if key in seen:
                continue
            seen.add(key)
            low = it["title"].lower()
            who = list(dict.fromkeys(BROKER_RE.findall(it["title"])))
            tg = TARGET_RE.search(it["title"])
            call = ("Buy" if re.search(r"\b(buy|outperform|overweight|accumulate|
                    "Sell" if re.search(r"\b(sell|underperform|underweight|reduce
                    "Hold" if re.search(r"\b(hold|neutral|market perform)\b", low
```

```
            if not (who or tg or call):
                continue
            it.update(who=who, target=int(tg.group(1).replace(",", "")) if tg and
                      call=call, view={"Buy": 1, "Sell": -1}.get(call, 0))
            out.append(it)
    out.sort(key=lambda i: (not i["who"], i["age_h"]))
    return out[:5]

def prio_of(s):
    if s.get("watch"):
        return 3
    f = s.get("f") or {}
    fl = sum(bool(f.get(k)) for k in ("ath_q_sales", "ath_q_profit", "ath_y_sales
    if fl >= 4:
        return 2
    if fl == 3 or (f.get("results") or {}).get("verdict") == "Excellent":
        return 1
    return 0

def bottleneck_scan():
    """Live shortage / export-curb / lead-time headlines for each chokepoint in B
    alarm = ["shortage", "ban", "curb", "restriction", "export control", "lead ti
             "tight", "constraint", "scarcity", "crunch", "delay", "sanction"]
    out = {}
    for b in BOTTLENECKS:
        items, seen = [], set()
        for q in b["queries"]:
            r = get("https://news.google.com/rss/search", params={"q": q + " when
            if r is not None:
                for it in feed_items(r.content, 10):
                    key = it["title"].lower()[:70]
                    if key in seen:
                        continue
                    seen.add(key)
                    it["alarm"] = any(w in it["title"].lower() for w in alarm)
                    items.append(it)
            time.sleep(0.5)
        n_alarm = sum(1 for i in items if i["alarm"])
        items.sort(key=lambda i: (not i["alarm"], i["age_h"]))
        out[b["id"]] = {"news": items[:8], "alarm": n_alarm, "heat": "High" if n_
    print("bottlenecks:", {k: v["heat"] for k, v in out.items()})
    return out

def pli_scan():
    out, seen = [], set()
    for q in PLI_QUERIES:
```

```
        r = get("https://news.google.com/rss/search", params={"q": q + " when:60d
        if r is not None:
            for it in feed_items(r.content, 10):
                key = it["title"].lower()[:70]
                if key not in seen:
                    seen.add(key)
                    it["amt"] = find_amt(it["title"])
                    out.append(it)
        time.sleep(0.5)
    out.sort(key=lambda i: (not i["amt"], i["age_h"]))
    return out[:15]

# ---------------------------------------------------------------------------
# Government / policy tracking
# ---------------------------------------------------------------------------
def _ctext(it, name):
    for c in it:
        if c.tag.split("}")[-1] == name:
            return (c.text or "").strip(), c
    return "", None

def feed_items(content, limit=12):
    """Generic RSS/Atom parser."""
    out = []
    try:
        root = ET.fromstring(content)
    except Exception:
        return out
    for it in root.iter():
        if it.tag.split("}")[-1] not in ("item", "entry"):
            continue
        title, _ = _ctext(it, "title")
        link, lc = _ctext(it, "link")
        if not link and lc is not None:
            link = lc.get("href", "")
        ds = _ctext(it, "pubDate")[0] or _ctext(it, "published")[0] or _ctext(it,
        try:
            dt = parsedate_to_datetime(ds)
        except Exception:
            try:
                dt = datetime.fromisoformat(ds.replace("Z", "+00:00"))
            except Exception:
                dt = datetime.now(timezone.utc)
        if dt.tzinfo is None:
            dt = dt.replace(tzinfo=timezone.utc)
        age_h = (datetime.now(timezone.utc) - dt).total_seconds() / 3600
```

```
        src = _ctext(it, "source")[0]
        if title and link.startswith("http"):
            out.append({"title": title, "link": link, "src": src, "age_h": round(
                        "ago": f"{int(age_h)}h ago" if age_h < 48 else f"{int(age
        if len(out) >= limit:
            break
    return out

def policy_item_score(it):
    low = it["title"].lower()
    s = min(3, sum(w in low for w in POLICY_POS)) - sum(w in low for w in POLICY_
    it["s"] = (s > 0) - (s < 0)
    weight = 2 if it["age_h"] <= 24 * 7 else 1 if it["age_h"] <= 24 * 30 else 0
    return s * weight

def tag_theme(title):
    low = title.lower()
    return next((th for th, kws in POLICY_KW.items() if any(k in low for k in kws

def policy_scan(extra_sectors=()):
    queries = dict(POLICY_QUERIES)
    for sct in extra_sectors:
        queries[sct] = [f"{sct} India government scheme Cabinet approves"]
    themes = {th: {"items": []} for th in queries}
    seen = set()
    def add(th, it):
        key = it["title"].lower()[:70]
        if key not in seen and it["age_h"] <= 24 * 30:
            seen.add(key); themes[th]["items"].append(it)
    for th, qs in queries.items():
        for q in qs:
            r = get("https://news.google.com/rss/search",
                    params={"q": q + " when:30d", "hl": "en-IN", "gl": "IN", "cei
            if r is not None:
                for it in feed_items(r.content, 8):
                    add(th, it)
            time.sleep(0.5)
    r = get(PIB_FEED)
    if r is not None:
        for it in feed_items(r.content, 60):
            th = tag_theme(it["title"])
            if th:
                it["src"] = it["src"] or "PIB"; add(th, it)
    letters = []
    for url in NEWSLETTER_FEEDS:
        r = get(url)
```

```
        if r is None:
            continue
        for it in feed_items(r.content, 10):
            it["src"] = it["src"] or re.sub(r"^https?://(www\.)?", "", url).split
            it["theme"] = tag_theme(it["title"]) or "General"
            policy_item_score(it); letters.append(it)
    for th, d in themes.items():
        score = sum(policy_item_score(it) for it in d["items"])
        d["items"].sort(key=lambda i: (-i["s"], i["age_h"]))
        d["items"] = d["items"][:6]
        d["score"] = score
        d["level"] = "High" if score >= 10 else "Medium" if score >= 4 else "Low"
    letters.sort(key=lambda i: i["age_h"])
    print("policy:", {k: v["level"] for k, v in themes.items()}, "newsletter item
    return {"themes": themes, "newsletters": letters[:25], "at": now_ist().strfti

def policy_view(p):
    if not p:
        return {"themes": [], "newsletters": [], "at": ""}
    th = [{"theme": k, "level": v["level"], "score": v["score"], "items": v["item
    th.sort(key=lambda x: -x["score"])
    return {"themes": th, "newsletters": p.get("newsletters", []), "at": p.get("a

# ---------------------------------------------------------------------------
# Sector cycle
# ---------------------------------------------------------------------------
def sector_cycle(stocks, policy=None):
    by = {}
    for s in stocks:
        for th in (s.get("areas") or ([s["theme"]] if s.get("theme") else [])):
            by.setdefault(th, []).append(s)
    out = []
    for theme, ss in by.items():
        fs = [s["f"] for s in ss]
        ms = [s["m"] for s in ss if s.get("m")]
        n = len(ss)
        earn = pct_true([f["ath_q_profit"] for f in fs])
        sales_ath = pct_true([f["ath_q_sales"] for f in fs])
        opm_up = pct_true([(f["opm_now"] or 0) > (f["opm_yr"] or 0) for f in fs i
        a200 = pct_true([m["price"] > m["dma200"] for m in ms if m.get("dma200")]
        r1, r3 = med([m["ret1m"] for m in ms]), med([m["ret3m"] for m in ms])
        r3v, r1v = r3 or 0, r1 or 0
        if n < 3:
            phase, stance, tone, pts = "Too few stocks to judge", "WAIT", "a", 2
        elif earn >= 40 and r3v > 0 and a200 >= 60:
            if r1v > 15:
```

```
                phase, stance, tone, pts = "Hot - extended upcycle", "FOCUS (don'
            else:
                phase, stance, tone, pts = "Upcycle - earnings and price both str
        elif earn >= 40:
            phase, stance, tone, pts = "Earnings at peak but price weakening (lat
        elif earn >= 20 and r3v > 0:
            phase, stance, tone, pts = "Early upcycle - earnings starting to buil
        elif earn >= 20:
            phase, stance, tone, pts = "Earnings improving but price not confirmi
        else:
            phase, stance, tone, pts = "Weak / out of favour", "AVOID", "r", 0
        pol = ((policy or {}).get("themes") or {}).get(theme, {})
        plevel = pol.get("level", "n/a")
        pts += {"High": 4, "Medium": 2}.get(plevel, 0)
        hay = (theme + " " + " ".join({s.get("tags", "") for s in ss})).lower()
        pli_s = [p for p in PLI_SCHEMES if any(k in hay for k in p["kw"])]
        info = THEME_INFO.get(theme, {})
        top = sorted(ss, key=lambda s: -(s.get("score") or 0))[:5]
        out.append({"theme": theme, "n": n, "phase": phase, "stance": stance, "to
                    "earn_ath_pct": round(earn), "sales_ath_pct": round(sales_ath
                    "above200_pct": round(a200), "ret1m": r1, "ret3m": r3,
                    "drivers": info.get("drivers", ""), "risks": info.get("risks"
                    "top": [s["sym"] for s in top],
                    "policy_level": plevel, "policy_top": (pol.get("items") or [{
                    "pli": [{"scheme": p["scheme"], "outlay_cr": p["outlay_cr"],
    out.sort(key=lambda x: -x["pts"])
    return out

# ---------------------------------------------------------------------------
# Scoring: Bullish / Watch / Avoid with reasons
# ---------------------------------------------------------------------------
def r1(x):
    return round(x, 1) if x is not None else None

def period_fresh(period, days=100):
    """True if the latest reported quarter ended within the last `days` days (so
    try:
        d = datetime.strptime("1 " + (period or "").strip(), "%d %b %Y") + timede
        return 0 <= (now_ist().replace(tzinfo=None) - d).days <= days
    except Exception:
        return False

def multibagger(s, f, m, rs, hv, mg, call, wc_ok, rk, adx_):
    """Early multibagger check: growing company + promoters/insiders buying + upt
    ins = s.get("insider") or []
    ibuy = any(i["kind"] == "Insider buying" and i["age_h"] <= 24 * 180 for i in
```

```
isell = any(i["kind"] == "Insider selling" and i["age_h"] <= 24 * 180 for i i
cg = f.get("cagr") or {}
p3, s3 = cg.get("profit3"), cg.get("sales3")
pch = ((hv or {}).get("ch") or {}).get("promoters") or {}
py, ph = pch.get("y"), pch.get("now")
pyoy, syoy = (rs or {}).get("profit_yoy"), (rs or {}).get("sales_yoy")
verdict = (rs or {}).get("verdict")
pe, mcap, pledged = f.get("pe"), f.get("mcap"), f.get("pledged") or 0
price, d200 = m.get("price"), m.get("dma200")
sc, why, risks = 0, [], []
growing = bool(verdict in ("Excellent", "Good") and pyoy is not None and pyoy
if p3 is not None:
    sc += 12 if p3 >= 30 else 9 if p3 >= 20 else 6 if p3 >= 15 else 0
if s3 is not None:
    sc += 6 if s3 >= 20 else 3 if s3 >= 12 else 0
sc += 7 if verdict == "Excellent" else 4 if verdict == "Good" else 0
sc += 8 if (mg or {}).get("verdict") == "Excellent" else 4 if (mg or {}).get(
tier_pts = sum(bool(f.get(k)) for k in ("ath_q_sales", "ath_q_profit", "ath_y
sc += {5: 7, 4: 5, 3: 2}.get(tier_pts, 0)
if ph is not None:
    sc += 5 if ph >= 60 else 4 if ph >= 50 else 2 if ph >= 40 else 0
prom_up = py is not None and py >= 0.3
prom_sell = py is not None and py <= -1
if py is not None:
    sc += 8 if py >= 1 else 5 if prom_up else -10 if py <= -2 else -4 if prom
if ibuy:
    sc += 8
    why.append("Promoters / directors buying shares (insider buying headlines
if isell:
    sc -= 10
    risks.append("Insider / promoter selling headlines")
if pledged >= 10:
    sc -= 8
    risks.append(f"Promoters pledged {pledged:.0f}% of shares")
if hv:
    inst = hv.get("inst")
    sc += 6 if inst == "Accumulating" else -4 if inst == "Reducing" else 0
    if any("New " in x["t"] for x in hv.get("signals", [])):
        sc += 3
if rk is not None:
    sc += 6 if rk >= 80 else 3 if rk >= 60 else -3 if rk < 30 else 0
uptrend = None if not (price and d200) else price > d200
if uptrend:
    sc += 3
if adx_ is not None and adx_ >= 25 and (m.get("pdi") or 0) > (m.get("mdi") or
    sc += 3
```

```
if s.get("m") and (m.get("vol_ratio") or 0) >= 1.5 and (m.get("ret5d") or 0)
    sc += 4
peg = (pe / p3) if (pe and p3 and p3 > 0) else None
if peg is not None:
    sc += 5 if peg <= 1 else 3 if peg <= 1.5 else 1 if peg <= 2 else -5 if pe
if pe and pe > 80:
    sc -= 5
if mcap:
    sc += 4 if mcap < 2000 else 3 if mcap < 10000 else 0 if mcap < 30000 else
if f.get("roce"):
    sc += 5 if f["roce"] >= 25 else 3 if f["roce"] >= 18 else 0
sc += 2 if wc_ok else -4 if wc_ok is False else 0
cfo_r, de_ = f.get("cfo_ratio"), f.get("de")
if cfo_r is not None:
    sc += 3 if cfo_r >= 0.8 else -4 if cfo_r < 0.4 else 0
if de_ is not None:
    sc += 2 if de_ <= 0.3 else -4 if de_ > 1.5 else 0
sc = max(0, min(100, round(sc)))
promoter_ok = bool(prom_up or ibuy)
not_selling = not prom_sell and not isell and pledged < 10
size_ok = None if not mcap else mcap <= 30000
val_ok = None if not pe else (pe <= 60 or (peg is not None and peg <= 2))
gates = [
    {"t": "Company is growing: latest quarter profit " + (f"{pyoy:+.0f}%" if
          (f"{syoy:+.0f}%" if syoy is not None else "n/a") + " year on year",
    {"t": "Long-run growth: profit " + (f"{p3:.0f}% a year over 3 years" if p
     "ok": None if p3 is None else p3 >= 15},
    {"t": "Promoters raising their stake or insiders buying" + (f" (promoter
     "ok": promoter_ok if (hv or s.get("insider")) else None},
    {"t": "Nobody selling: promoters not reducing, no pledged shares, no insi
    {"t": "Price in an uptrend (above its 200-day average)", "ok": uptrend},
    {"t": "Valuation not stretched" + (f" (P/E {pe:.0f}" + (f", PEG {peg:.1f}
    {"t": "Room to grow" + (f": market cap Rs {mcap:,.0f} Cr" if mcap else ""
    {"t": "Working capital healthy (debtor and inventory days not rising)", "
    {"t": "Profit backed by cash flow" + (f" (operating cash flow {cfo_r * 10
     "ok": None if cfo_r is None else cfo_r >= 0.6},
    {"t": "Debt manageable" + (f" (debt to equity {de_:.2f})" if de_ is not N
    {"t": "Tradable: not flagged AVOID", "ok": call != "AVOID"},
]
must = [growing, promoter_ok, not_selling, uptrend is not False, size_ok is n
        call != "AVOID", wc_ok is not False, cfo_r is None or cfo_r >= 0.4, d
if sc >= 70 and all(must) and hv:
    mcall = "MULTIBAGGER BUY"
elif sc >= 55 and growing and not_selling and call != "AVOID":
    mcall = "STRONG WATCH"
else:
```

```
        mcall = ""
    for g in gates:
        if g["ok"] is True:
            why.append(g["t"])
    return {"score": sc, "call": mcall, "gates": gates, "why": why[:6], "risks":
            "insider_buy": ibuy}

def build_view(s, cyc):
    f, m, news = s["f"], s.get("m") or {}, s.get("news", [])
    why, risks, score = [], [], 0
    aths = [("ath_q_sales", "quarterly sales"), ("ath_q_profit", "quarterly profi
            ("ath_y_sales", "annual sales"), ("ath_y_profit", "annual profit")]
    got = [l for k, l in aths if f.get(k)]
    miss = [l for k, l in aths if not f.get(k)]
    score += 8 * len(got)
    if len(got) == 4:
        why.append("All-time-high sales AND profit (quarterly + annual)")
    elif got:
        why.append("Record " + ", ".join(got))
    if miss:
        risks.append("Not at record: " + ", ".join(miss))
    flags = sum(bool(f.get(k)) for k, _ in aths) + bool(f.get("profit_breakout"))
    tier = "A" if flags == 5 else "B" if flags == 4 else "C" if flags == 3 else "
    if f.get("profit_breakout"):
        score += 8
        why.append(f"Profit breakout: +{f['profit_yoy']:.0f}% YoY and above the l
    elif f.get("profit_yoy") is not None:
        risks.append(f"No profit breakout (qtr profit {f['profit_yoy']:+.0f}% YoY
    if f.get("sales_yoy") is not None and f["sales_yoy"] >= 15:
        why.append(f"Sales growing {f['sales_yoy']:.0f}% YoY")
    if f.get("opm_now") is not None and f.get("opm_yr") is not None:
        if f["opm_now"] > f["opm_yr"]:
            why.append(f"Margins expanding ({f['opm_yr']:.0f}% to {f['opm_now']:.
        else:
            risks.append(f"Margins shrinking ({f['opm_yr']:.0f}% to {f['opm_now']
    if f.get("roce") and f["roce"] >= 20:
        score += 5
        why.append(f"ROCE {f['roce']:.0f}% - strong capital efficiency (moat prox

    hard_avoid = False
    wc_ok = f.get("wc_ok")
    fm = lambda a, b: f"{a:.0f} to {b:.0f}" if a is not None and b is not None el
    if wc_ok is True:
        why.append(f"Working capital healthy - debtor days {fm(f.get('debtor_prev
                   f"inventory days {fm(f.get('inv_prev'), f.get('inv_days'))} (n
    elif wc_ok is False:
```

```
    score -= 6
    risks.append(f"Working capital stretching - debtor days {fm(f.get('debtor
                 f"inventory days {fm(f.get('inv_prev'), f.get('inv_days'))}
if m:
    price, d50, d200, hi = m["price"], m.get("dma50"), m.get("dma200"), m.get
    gap = (hi - price) / hi * 100 if hi else None
    if d200 and price > d200:
        if gap is not None and gap <= NEAR_HIGH_PCT:
            score += 12
            why.append(f"Price {gap:.1f}% from 52-wk high, above 200-DMA - br
        else:
            score += 6
            why.append("Above 200-DMA - uptrend intact")
        if d50 and price > d50:
            score += 4
        if price / d200 - 1 > 0.40:
            score -= 6
            risks.append(f"Extended: {100*(price/d200-1):.0f}% above 200-DMA
    elif d200:
        if m.get("dma_state") in ("golden_cross", "converging_up", "squeeze")
            risks.append("Just under 200-DMA - needs to close above it to con
        else:
            hard_avoid = True
            risks.append("Below 200-DMA - trend not confirmed")
    st_ = m.get("dma_state")
    if st_ in ("golden_cross", "converging_up") and d200 and price >= d200 *
        score += 6
        gp = m.get("dma_gap")
        why.append("50-DMA meeting 200-DMA" + (f" (gap {gp:+.1f}%)" if gp is
                   + (f", golden cross {m['cross_days']} sessions ago" if st_
                   + " - classic trend-turn zone")
    elif st_ == "death_cross":
        score -= 5
        risks.append("50-DMA recently crossed below 200-DMA (death cross) - t
    vr, day = m.get("vol_ratio"), m.get("day", 0)
    if vr and vr >= 1.5 and day > 0:
        score += 10
        why.append(f"Volume {vr:.1f}x the 20-day average with price up {day:.
    elif vr and vr >= 1.2 and day > 0:
        score += 5
        why.append(f"Volume {vr:.1f}x average on an up day")
    elif vr and vr >= 1.5 and day < -1:
        score -= 6
        risks.append(f"Heavy volume ({vr:.1f}x) on a {day:.1f}% fall - sellin
    t = m.get("turnover_cr")
    if t is not None and t < MIN_TURNOVER_CR:
```

```
        score -= 5
        risks.append(f"Thin liquidity (~Rs {t:.1f} Cr/day) - hard to exit")
        if t < 1:
            hard_avoid = True
else:
    risks.append("No live price data yet")
pe = f.get("pe")
if pe and pe > 70:
    score -= 5
    risks.append(f"Rich valuation (P/E {pe:.0f})")

net = sum(n["s"] for n in news)
score += max(-10, min(10, net * 4))
pos = next((n for n in news if n["s"] > 0), None)
neg = next((n for n in news if n["s"] < 0), None)
if pos:
    why.append("News: " + pos["title"][:110])
if neg:
    risks.append("News: " + neg["title"][:110])
if net <= -2:
    hard_avoid = True

cc, cr, bk, pl = s.get("concall") or {}, s.get("credit") or {}, s.get("broker
ob_text = cc.get("ob_text", "")
if cc.get("ob_cr") and f.get("y_sales"):
    yrs = cc["ob_cr"] / f["y_sales"]
    ob_text = f"{ob_text} = {yrs:.1f}x annual sales"
    if yrs >= 1.5:
        score += 5
        why.append(f"Strong order book: {yrs:.1f}x annual sales (revenue visi
    elif yrs < 0.5:
        risks.append(f"Thin order book: only {yrs:.1f}x annual sales")
if cc.get("tone") == "Positive":
    score += 3
    why.append("Latest concall: management tone confident / positive")
elif cc.get("tone") == "Cautious":
    score -= 4
    risks.append("Latest concall: management tone cautious (headwinds / delay
if cc.get("guidance") and cc.get("tone") != "Cautious":
    why.append("Management outlook: " + cc["guidance"][0][:130])
rt = cr.get("rating", "")
if rt:
    if re.match(r"(AAA|AA)", rt) and "Negative" not in rt:
        score += 3
        why.append(f"Credit rating {rt} ({cr.get('agency', '')}) - strong bal
    elif "Negative" in rt or re.match(r"(BBB|BB)", rt):
```

```
        score -= 4
        risks.append(f"Credit rating {rt} ({cr.get('agency', '')}) - weaker c
bnet = sum(b.get("view", 0) for b in bk)
if bnet > 0:
    score += 2
    why.append("Brokerage view positive: " + bk[0]["title"][:100])
elif bnet < 0:
    score -= 3
    risks.append("Brokerage view negative: " + bk[0]["title"][:100])
if pl:
    score += 2
    why.append("PLI link: " + pl[0]["title"][:100])

rs, hv, inv = f.get("results"), s.get("holding"), s.get("invest") or []
if rs:
    bits = [f"{lab} {rs[k]:+.0f}% YoY" for lab, k in (("sales", "sales_yoy"),
    rtxt = f"{rs.get('period', '')}: {rs['verdict']}" + (" (" + ", ".join(bit
    if rs["verdict"] == "Excellent":
        score += 3
        why.append("Latest quarter " + rtxt)
    elif rs["verdict"] == "Good":
        score += 1
        why.append("Latest quarter " + rtxt)
    elif rs["verdict"] in ("Weak", "Loss-making"):
        score -= 5
        risks.append("Latest quarter " + rtxt)
if hv:
    score += 3 if hv["inst"] == "Accumulating" else -3 if hv["inst"] == "Redu
    for sg_ in hv["signals"]:
        (why if sg_["tone"] == "g" else risks).append(sg_["t"])
if (f.get("pledged") or 0) >= 10:
    score -= 4
    risks.append(f"Promoters have pledged {f['pledged']:.0f}% of their shares
for it in inv:
    if it["kind"].startswith("Preferential") and it["age_h"] <= 24 * 120:
        risks.append("Fund-raise headline - check who subscribed and at what
        break
for it in inv:
    if it["kind"] == "Big investor" and it["age_h"] <= 24 * 60:
        why.append("Big-investor headline: " + it["title"][:100])
        break

mg, cg = f.get("margin"), f.get("cogs")
if mg:
    txt = ""
    if cg and cg.get("now") is not None and cg.get("yr") is not None:
```

```
        txt = f" (COGS {cg['yr']:.0f}% to {cg['now']:.0f}% of sales)"
    elif mg.get("cost_now") is not None and mg.get("cost_yr") is not None:
        txt = f" (costs {mg['cost_yr']:.0f}% to {mg['cost_now']:.0f}% of sale
    if mg["verdict"] == "Excellent":
        score += 5
        why.append("Cost quality excellent: costs falling, margin expanding,
    elif mg["verdict"] == "Good":
        score += 2
        why.append("Margins improving and operating profit growing" + txt)
    elif mg["verdict"] == "Weak":
        score -= 5
        risks.append("Cost quality weak: margin shrinking or operating profit
adx_, pdi_, mdi_ = m.get("adx"), m.get("pdi"), m.get("mdi")
if adx_ is not None:
    if adx_ >= 30 and (pdi_ or 0) > (mdi_ or 0):
        score += 5
        why.append(f"ADX {adx_:.0f} (above 30 = excellent trend strength) wit
    elif adx_ >= 30:
        score -= 5
        risks.append(f"ADX {adx_:.0f}: strong DOWN trend (sellers in control)
    elif adx_ >= 20 and (pdi_ or 0) > (mdi_ or 0):
        score += 2
        why.append(f"ADX {adx_:.0f}: uptrend building")
    elif adx_ < 20:
        risks.append(f"ADX {adx_:.0f}: no clear trend yet")
rk = m.get("rs_rank")
if rk is not None:
    if rk >= 80:
        score += 5
        why.append(f"Relative strength rank {rk}/99: outperforming most stock
    elif rk >= 60:
        score += 2
    elif rk < 30:
        score -= 4
        risks.append(f"Weak relative strength (rank {rk}/99)")
mc_ = f.get("mcap")
if mc_ and mc_ < 500:
    risks.append(f"Micro cap (Rs {mc_:.0f} Cr): thin trading, higher risk - k
accum_info = None
flat = [lab for lab, k_, lim in (("2 days", "ret2d", 2), ("1 week", "ret5d",
        if m.get(k_) is not None and m[k_] < lim]
vsurge = max(m.get("vol_ratio") or 0, m.get("vol_max3") or 0)
if rs and rs["verdict"] in ("Excellent", "Good") and flat and vsurge >= 1.5 a
        (not m.get("dma200") or m["price"] >= m["dma200"] * 0.97):
    score += 8
    accum_info = {"flat": flat, "vol": vsurge}
```

```
    why.append(f"Quiet accumulation: {rs['verdict']} results, price flat over
               f"volume {vsurge:.1f}x its 20-day average - big institutions m

cfo_r, de_ = f.get("cfo_ratio"), f.get("de")
if cfo_r is not None:
    if cfo_r >= 0.8:
        score += 3
        why.append(f"Profits backed by cash: operating cash flow is {cfo_r *
    elif cfo_r < 0.4:
        score -= 4
        risks.append(f"Weak cash conversion: operating cash flow only {cfo_r
if de_ is not None:
    if de_ <= 0.3:
        score += 2
        why.append(f"Low debt (debt to equity {de_:.2f})")
    elif de_ > 1.5:
        score -= 4
        risks.append(f"High debt (debt to equity {de_:.1f})")
cx = f.get("capex") or {}
ath4 = bool(f.get("ath_q_sales") and f.get("ath_q_profit") and f.get("ath_y_s
athq = bool(f.get("ath_q_sales") and f.get("ath_q_profit"))
athy = bool(f.get("ath_y_sales") and f.get("ath_y_profit"))
hi52_, pr52_ = m.get("hi52"), m.get("price")
dd = ((hi52_ - pr52_) / hi52_ * 100) if (hi52_ and pr52_ and hi52_ > 0) else
pn_, py_ = (rs or {}).get("profit"), (rs or {}).get("profit_yoy")
pa_ = (pn_ / (1 + py_ / 100)) if (pn_ is not None and py_ is not None and py_
expanding = bool(athq and (cx.get("fa_yoy") or -99) >= 15 and (cx.get("cfo_no
                 and (cx.get("cfo_yoy") is None or cx["cfo_yoy"] > 0))
if expanding:
    score += 3
    why.append(f"Capacity expanding with record results: fixed assets {cx['fa
               + (f", plant and machinery {cx['pm_yoy']:+.0f}%" if cx.get("pm
               + (f", work in progress {cx['cwip_pct']:.0f}% of fixed assets"
               + (f", operating cash flow {cx['cfo_yoy']:+.0f}%" if cx.get("c
ords = [i for i in (s.get("orders") or []) if i.get("age_h", 0) <= 24 * 180]
flt = [x for x in ((s.get("docs") or {}).get("filings") or []) if re.search(r
cr_news = sum({round(i["amt_cr"], 1) for i in ords if i.get("amt_cr")})
cr_file = sum({round(amt_cr_of(x["text"]), 1) for x in flt if amt_cr_of(x["te
ocr = max(cr_news, cr_file)
ysales = f.get("y_sales")
opct = (ocr / ysales * 100) if (ocr and ysales and ysales > 0) else None
ordd = {"n": len(ords) + len(flt), "cr": ocr or None, "pct": opct, "files": f
        "cust": list(dict.fromkeys(n_ for i in ords for n_ in (i.get("cust")
if opct is not None:
    if opct >= 25:
        score += 4
```

```
        why.append(f"Big order wins: about Rs {ocr:,.0f} Cr announced in 6 mo
                   + (f" (customers: {', '.join(ordd['cust'])})" if ordd["cus
    elif opct >= 10:
        score += 2
        why.append(f"Order wins of about Rs {ocr:,.0f} Cr in 6 months = {opct
if cc.get("certs"):
    score += 2
    why.append("Quality approvals mentioned on the concall: " + ", ".join(cc[
if cc.get("quals"):
    score += 2
    why.append("Vendor-qualification / long-term-agreement language on the co
if (cc.get("top_cust_pct") or 0) >= 40:
    risks.append(f"Customer concentration: top customers are about {cc['top_c

cy = next((c for c in cyc if c["theme"] == s["theme"]), None)
if cy:
    score += cy["pts"]
    (why if cy["tone"] == "g" else risks if cy["tone"] == "r" else why).appen
    if cy.get("policy_level") == "High":
        why.append("Policy tailwind: " + (cy.get("policy_top") or "strong gov

score = max(0, min(100, round(score)))
if hard_avoid or score < 45 or (tier == "-" and score < 55):
    verdict = "Avoid"
elif score >= 70 and tier in ("A", "B") and m and wc_ok is not False:
    verdict = "Bullish"
else:
    verdict = "Watch"
pr_, d50_, d200_ = m.get("price"), m.get("dma50"), m.get("dma200")
ext = bool(pr_ and ((d50_ and pr_ > d50_ * 1.10) or (d200_ and pr_ / d200_ -
if verdict == "Avoid":
    call = "AVOID"
elif verdict == "Bullish":
    call = "HOLD" if ext else "BUY"
else:
    call = "HOLD" if (tier in ("A", "B") and score >= 60 and ext) else "WAIT"

plan = {"stop": None, "maxw": 0, "entry": ""}
if m and verdict != "Avoid":
    d50 = m.get("dma50") or m["price"] * 0.92
    if m.get("atr"):
        plan["stop"] = round(max(m["price"] - 2 * m["atr"], m["price"] * 0.85
    else:
        plan["stop"] = round(max(d50 * 0.98, m["price"] * 0.88), 1)
    risk_ = m["price"] - plan["stop"]
    plan["t1"], plan["t2"] = round(m["price"] + 2 * risk_, 1), round(m["price
```

```
    plan["risk_pct"] = round(100 * risk_ / m["price"], 1)
    plan["maxw"] = (5 if tier == "A" else 4) if verdict == "Bullish" else 2
    if (f.get("mcap") or 1e9) < 1000:
        plan["maxw"] = min(plan["maxw"], 3)
    if m["price"] > d50 * 1.08:
        plan["entry"] = f"Extended above 50-DMA - wait for a dip toward Rs {d
    elif verdict == "Bullish":
        plan["entry"] = "Start 1/3 position now, add 1/3 on a volume-backed n
    else:
        plan["entry"] = "On the watchlist. Wait for volume above 1.5x average
    if call == "HOLD":
        plan["entry"] = f"Strong but extended: keep what you own, do not add.
custs = {c["name"]: dict(c) for c in cc.get("customers", [])}
for it in s.get("orders") or []:
    for nm_ in it.get("cust", []):
        if nm_ not in custs:
            custs[nm_] = {"name": nm_, "region": CUSTOMERS.get(nm_, ""), "n":
customers = sorted(custs.values(), key=lambda c: -c["n"])[:8]
trg = []
def _t(kind, w, text):
    trg.append({"k": kind, "w": w, "t": text})
if ordd["pct"] is not None and ordd["pct"] >= 10:
    _t("order", 3, f"New orders ~Rs {ordd['cr']:,.0f} Cr = {ordd['pct']:.0f}%
elif any(i.get("age_h", 9e9) <= 24 * 30 for i in ords):
    _t("order", 1, "Fresh order headline" + (f" ({ordd['cust'][0]})" if ordd[
if any(i["kind"] == "Insider buying" and i["age_h"] <= 24 * 60 for i in (s.ge
    _t("insider", 3, "Promoter / insider buying")
pch_ = ((hv or {}).get("ch") or {}).get("promoters") or {}
if (pch_.get("y") or 0) >= 1:
    _t("insider", 2, f"Promoter stake up {pch_['y']:.1f} pts in a year")
if hv:
    if any("New " in x["t"] for x in hv.get("signals", [])):
        _t("inst", 3, "New mutual fund / FII entry")
    elif hv.get("inst") == "Accumulating":
        _t("inst", 2, "Institutions accumulating")
if any(i["kind"] == "Big investor" and i["age_h"] <= 24 * 30 for i in inv):
    _t("deal", 2, "Big-investor / block-deal headline")
if any(i["kind"].startswith("Preferential") and i["age_h"] <= 24 * 60 for i i
    _t("deal", 1, "Preferential / fund-raise headline (check the terms)")
bpos = next((b for b in bk if b.get("view", 0) > 0 and b.get("age_h", 9e9) <=
if bpos:
    _t("broker", 2, "Brokerage positive" + (f": {bpos['who'][0]}" if bpos.get
hi_, pr_x = m.get("hi52"), m.get("price")
vmax = max(m.get("vol_ratio") or 0, m.get("vol_max3") or 0)
if hi_ and pr_x and pr_x >= hi_ * 0.98 and vmax >= 1.5:
    _t("brk", 3, f"Breakout near 52-week high on {vmax:.1f}x volume")
```

```
    if (m.get("dma_state") or "").startswith("Golden") and (m.get("cross_days") o
        _t("brk", 2, "Fresh golden cross (50-day above 200-day)")
    if rs and rs["verdict"] in ("Excellent", "Good") and period_fresh(rs.get("per
        _t("res", 3 if rs["verdict"] == "Excellent" else 1, f"Fresh results ({rs[
    if accum_info:
        _t("accum", 3, f"Quiet accumulation: volume {accum_info['vol']:.1f}x, pri
    if cy and cy.get("policy_level") == "High":
        _t("pol", 1, "Government focus on this sector")
    if pl:
        _t("pol", 2, "PLI scheme link")
    tscore = sum(x["w"] for x in trg)
    mb = multibagger(s, f, m, rs, hv, mg, call, wc_ok, rk, adx_)
    sym = s["sym"]
    return {
        "sym": sym, "name": s["name"], "theme": s["theme"], "tier": tier, "flags"
        "verdict": verdict, "price": m.get("price"), "day": m.get("day"), "vol_ra
        "turnover_cr": m.get("turnover_cr"), "asof": m.get("asof"), "pe": pe, "ro
        "mcap": f.get("mcap"), "period": f.get("period"), "ret1m": m.get("ret1m")
        "why": why, "risks": risks, "news": news, "plan": plan, "net_news": net,
        "concall": cc, "credit": cr, "broker": bk, "pli": pl, "ob_text": ob_text,
        "dma_gap": m.get("dma_gap"), "dma_state": m.get("dma_state"), "cross_days
        "wc": {k: f.get(k) for k in ("debtor_days", "debtor_prev", "inv_days", "i
        "results": rs, "holding": hv, "invest": inv, "filings": (s.get("docs") or
        "watch": bool(s.get("watch")), "pledged": f.get("pledged"),
        "adx": adx_, "pdi": pdi_, "mdi": mdi_, "rs_rank": rk, "rs_nifty6m": m.get
        "margin": mg, "cogs": cg, "accum": accum_info, "ret2d": m.get("ret2d"), "
        "call": call, "areas": s.get("areas", []), "mb": mb, "ord": ordd, "trigge
        "vol_max3": m.get("vol_max3"), "ath4": ath4, "athq": athq, "athy": athy,
        "targets": [x["area"] for x in cc.get("targets", [])], "about": (s.get("a
        "customers": customers, "orders": s.get("orders", []),
        "links": {"screener": s["url"], "chart": f"https://in.tradingview.com/cha
                  "news": "https://news.google.com/search?q=" + requests.utils.qu
    }

# ---------------------------------------------------------------------------
# Alerts
# ---------------------------------------------------------------------------
def send_alert(text):
    tg, chat = os.getenv("TELEGRAM_TOKEN"), os.getenv("TELEGRAM_CHAT_ID")
    if tg and chat:
        try:
            requests.post(f"https://api.telegram.org/bot{tg}/sendMessage",
                          data={"chat_id": chat, "text": text[:4000], "disable_we
        except requests.RequestException:
            pass
    eu, ep, eto = os.getenv("EMAIL_USER"), os.getenv("EMAIL_PASS"), os.getenv("EM
```

```
    if eu and ep and eto:
        try:
            m = EmailMessage()
            m["Subject"], m["From"], m["To"] = "India Moat Screener alert", eu, e
            m.set_content(text)
            with smtplib.SMTP_SSL("smtp.gmail.com", 465) as srv:
                srv.login(eu, ep); srv.send_message(m)
        except Exception as e:
            print("email failed:", e)

def do_alerts(views, data):
    done = set(data.get("alerted", []))
    msgs = []
    today = now_ist().strftime("%Y-%m-%d")
    for v in views:
        k = f"{v['sym']}|multibagger"
        if v["mb"]["call"] == "MULTIBAGGER BUY" and k not in done:
            msgs.append(f"MULTIBAGGER BUY: {v['name']} ({v['theme']})\nMulti-bagg
                        + "; ".join(v["mb"]["why"][:3]) + f"\n{v['links']['screen
            done.add(k)
        o_ = v.get("ord") or {}
        k = f"{v['sym']}|ord|{int(o_.get('cr') or 0)}"
        if o_.get("pct") and o_["pct"] >= 25 and (v.get("mcap") or 1e9) < 5000 an
            msgs.append(f"BIG ORDERS: {v['name']} announced about Rs {o_['cr']:,.
                        + (f" (customers: {', '.join(o_['cust'])})" if o_.get("cu
            done.add(k)
        ks_ = {x["k"] for x in v.get("triggers", [])}
        k = f"{v['sym']}|trig|{today}"
        if v.get("tscore", 0) >= 8 and len(ks_) >= 3 and v["call"] != "AVOID" and
            msgs.append(f"TRIGGERS: {v['name']} ({v['call']}, score {v['score']})
                        + "; ".join(x["t"] for x in sorted(v["triggers"], key=lam
            done.add(k)
        if v["verdict"] == "Avoid":
            continue
        k = f"{v['sym']}|tier{v['tier']}"
        if v["tier"] == "A" and k not in done and v["score"] >= ALERT_MIN_SCORE:
            msgs.append(f"NEW all-rule match: {v['name']} ({v['theme']})\nScore {
                        f"{v['why'][0] if v['why'] else ''}\n{v['links']['screene
        if v["score"] >= ALERT_MIN_SCORE or v["tier"] in ("A", "B"):
            for n in v["news"]:
                if n["s"] != 0 and n["age_h"] <= 24:
                    k = f"{v['sym']}|news|{hashlib.md5(n['title'].encode()).hexdi
                    if k not in done:
                        icon = "NEWS (+)" if n["s"] > 0 else "NEWS (-)"
                        msgs.append(f"{icon} {v['name']}: {n['title']}\n{n['link'
            vr = v.get("vol_ratio")
```

```
            k = f"{v['sym']}|vol|{today}"
            if vr and vr >= 2 and (v.get("day") or 0) > 1 and k not in done:
                msgs.append(f"VOLUME SPIKE {v['name']}: {vr:.1f}x avg volume, pri
                done.add(k)
    data["alerted"] = list(done)[-3000:]
    if msgs:
        send_alert("\n\n".join(msgs))
    print(f"alerts sent: {len(msgs)}")

# ---------------------------------------------------------------------------
# Pipelines
# ---------------------------------------------------------------------------
def load_data():
    try:
        return json.load(open(DATA_FILE))
    except Exception:
        return None

RAW_SHARDS = 16
KEEP_KEYS = ("m", "news", "concall", "credit", "broker", "pli", "invest", "inside

def safe_sym(sym):
    return re.sub(r"[^A-Za-z0-9_.-]", "_", sym)

def load_raw():
    raw = {}
    for i in range(RAW_SHARDS):
        try:
            raw.update(json.load(open(f"{DATA_DIR}/raw/{i:02d}.json")))
        except Exception:
            pass
    return raw

def save_raw(raw):
    os.makedirs(f"{DATA_DIR}/raw", exist_ok=True)
    shards = [{} for _ in range(RAW_SHARDS)]
    for sym, s in raw.items():
        shards[int(hashlib.md5(sym.encode()).hexdigest(), 16) % RAW_SHARDS][sym]
    for i, sh in enumerate(shards):
        json.dump(clean(sh), open(f"{DATA_DIR}/raw/{i:02d}.json", "w"), separator

def load_state():
    try:
        return json.load(open(DATA_FILE))
    except Exception:
        return {}
```

```
def save_state(state):
    os.makedirs(DATA_DIR, exist_ok=True)
    json.dump(clean(state), open(DATA_FILE, "w"), separators=(",", ":"))

def clean(o):
    # JSON cannot hold NaN / Infinity, so turn them into null
    if isinstance(o, float):
        return o if o == o and o not in (float("inf"), float("-inf")) else None
    if isinstance(o, dict):
        return {k: clean(v) for k, v in o.items()}
    if isinstance(o, (list, tuple)):
        return [clean(v) for v in o]
    return o

_PACE = {"delay": 1.0}

def fetch_company(url):
    """Returns the page html, "" if the page does not exist, None if Screener kee
    When Screener says 'too many requests' we wait, slow our pace, and try again.
    for i in range(5):
        try:
            r = S.get(url, timeout=30)
        except requests.RequestException:
            time.sleep(5)
            continue
        if r.status_code in (429, 403) or r.status_code >= 500:
            try:
                ra = int(r.headers.get("Retry-After", 0))
            except ValueError:
                ra = 0
            wait = min(600, ra or 30 * (2 ** i))
            _PACE["delay"] = min(6.0, _PACE["delay"] * 1.5)
            print(f"Screener said HTTP {r.status_code}: waiting {wait}s and slowi
            time.sleep(wait)
            continue
        _PACE["delay"] = max(1.0, _PACE["delay"] * 0.97)
        return r.text if (r.status_code == 200 and "top-ratios" in r.text) else "
    return None

def exchange_universe():
    """Fallback list of every NSE-listed company, used when Screener's own listin
    import csv
    rows = []
    try:
        r = requests.get("https://nsearchives.nseindia.com/content/equities/EQUIT
```

```
        if r.status_code == 200:
            for rec in list(csv.reader(r.text.splitlines()))[1:]:
                if len(rec) > 2 and rec[0].strip() and rec[2].strip() in ("EQ", "
                    sym = rec[0].strip()
                    rows.append({"name": rec[1].strip() or sym, "url": f"{BASE}/c
    except Exception as e:
        print("NSE list failed:", e)
    return rows

def short_flags(s):
    f = s["f"]
    return sum(bool(f[k]) for k in ("ath_q_sales", "ath_q_profit", "ath_y_sales",

def full_scan():
    t0 = time.time()
    budget = float(os.getenv("BUDGET_MIN") or BUDGET_MIN) * 60
    dl = lambda frac: t0 + budget * frac
    scan_note = ""
    state, raw = load_state(), load_raw()
    login()
    rows = run_screen()
    print(f"Screener universe: {len(rows)} companies; {len(raw)} already saved")
    if len(rows) < 1500:
        have_u = {r_["url"] for r_ in rows}
        extra_rows = [x for x in exchange_universe() if x["url"] not in have_u]
        print(f"Screener's own listing is short - adding {len(extra_rows)} compan
        rows = rows + extra_rows
    watch = sorted(set(EXTRA_WATCHLIST) | set(state.get("watch", [])) |
                   {x for x in re.split(r"[,\s]+", os.getenv("WATCH", "").upper()
    by_sym = {}
    for r_ in rows:
        mm = re.search(r"/company/([^/]+)/", r_["url"])
        if mm:
            by_sym.setdefault(mm.group(1), r_)
    have = {k.upper() for k in by_sym}
    for w in watch:
        if w not in have:
            by_sym[w] = {"name": w, "url": f"{BASE}/company/{w}/consolidated/", "
    wset = set(watch)
    now = time.time()
    for sym in by_sym:
        if sym in raw:
            raw[sym]["seen_at"] = now
    for sym in list(raw):
        if sym not in by_sym and now - raw[sym].get("seen_at", now) > 30 * 86400:
            del raw[sym]
```

```
# ---- phase 1: financials from Screener (new and stale stocks first) ----
def due(sym):
    o = raw.get(sym)
    if not o:
        return True
    return now - o.get("fund_at", 0) > 86400 * (1 if prio_of(o) >= 1 else FUN
try:
    pri_urls = {x["url"] for x in screen_rows(PRIORITY_QUERY, 30)[0]}
except Exception:
    pri_urls = set()
print(f"priority list (fast-growing companies, loaded first): {len(pri_urls)}
todo = sorted([sy for sy in by_sym if due(sy)],
              key=lambda sy: (sy in raw, 0 if by_sym[sy]["url"] in pri_urls e
                              -prio_of(raw[sy]) if sy in raw else 0, raw.get(
print(f"phase 1: {len(todo)} stocks need fresh financials")
done = blocked = 0
for sym in todo:
    if time.time() > dl(0.62):
        print("phase 1: time slice used up, will continue next run")
        break
    r_ = by_sym[sym]
    html = fetch_company(r_["url"])
    if html is None:
        blocked += 1
        if blocked >= 2:
            scan_note = "Screener slowed us down; the rest loads on the next
            print("Screener keeps refusing requests - stopping page fetches f
            break
        continue
    blocked = 0
    if not html and (r_.get("watch") or r_.get("guess")):
        html = fetch_company(r_["url"].replace("/consolidated/", "/")) or ""
    if not html:
        continue
    try:
        s = parse_company(html, r_)
    except Exception as e:
        print("parse failed", sym, e)
        continue
    if (r_.get("watch") or r_.get("guess")) and s["f"]["q_sales"] is None:
        if r_.get("watch"):
            print("watchlist symbol not found:", sym)
        continue
    if short_flags(s) >= 3 or s.get("watch"):
        s["f"]["cogs"] = fetch_cogs(html, r_["url"])
```

```
        time.sleep(0.6)
        pmx = fetch_pm(html, r_["url"])
        if pmx:
            s["f"].setdefault("capex", {}).update(pmx)
        time.sleep(0.6)
    o = raw.get(sym, {})
    for k in KEEP_KEYS:
        if k in o:
            s[k] = o[k]
    if sym.upper() in wset:
        s["watch"] = True
    s["fund_at"], s["seen_at"] = time.time(), now
    raw[sym] = s
    done += 1
    time.sleep(_PACE["delay"])
print(f"phase 1 done: {done} stocks refreshed; {len(raw)} of {len(by_sym)} un
save_raw(raw)

# ---- phase 2: price, volume, 50/200 DMA ----
n2 = 0
for s in sorted(raw.values(), key=lambda x: (-prio_of(x), (x.get("m") or {}).
    if time.time() > dl(0.74):
        break
    if prio_of(s) < 2 and time.time() - (s.get("m") or {}).get("ts", 0) < 16
        continue
    m = market_metrics(s["sym"])
    if m:
        m["ts"] = time.time()
        s["m"] = m
    n2 += 1
    time.sleep(0.35)
print(f"phase 2 done: market data for {n2} stocks")
save_raw(raw)

# ---- phase 3: news, PLI, big investors, orders, brokers (shortlisted stocks
def deep_due(s):
    return time.time() - s.get("deep_at", 0) > {3: 0.5, 2: 1, 1: 3, 0: 10}[pr
n3 = 0
for s in sorted([x for x in raw.values() if deep_due(x)], key=lambda x: (-pri
    if time.time() > dl(0.88):
        break
    try:
        s["news"] = get_news(s["name"]); time.sleep(0.5)
        s["pli"] = pli_stock(s["name"]); time.sleep(0.4)
        s["invest"] = investor_news(s["name"])
        s["insider"] = insider_news(s["name"]); time.sleep(0.4)
```

```
            s["orders"] = orders_news(s["name"]); time.sleep(0.4)
            s["broker"] = broker_view(s["name"]); time.sleep(0.4)
            s["deep_at"] = time.time()
            n3 += 1
        except Exception as e:
            print("deep scan failed", s["name"], e)
    print(f"phase 3 done: news/investor/orders/brokers for {n3} stocks")
    save_raw(raw)

    # ---- phase 4: concall transcripts and credit ratings ----
    n4 = 0
    for s in sorted(raw.values(), key=lambda x: -prio_of(x)):
        if time.time() > dl(0.95) or n4 >= ENRICH_PER_RUN:
            break
        d = s.get("docs") or {}
        key = (d.get("transcript") or "") + "|" + ((d.get("ratings") or [{}])[0].
        if key == "|" or s.get("enrich_key") == key:
            continue
        try:
            enrich(s, s)
            s["enrich_tries"] = s.get("enrich_tries", 0) + 1
            if s.get("concall") or s.get("credit") or s["enrich_tries"] >= 3:
                s["enrich_key"], s["enrich_tries"] = key, 0
            n4 += 1
        except Exception as e:
            print("enrich failed", s["name"], e)
    print(f"phase 4 done: concalls / ratings read for {n4} stocks")

    # ---- policy and PLI headlines ----
    extra = sorted({s["sector"] for s in raw.values() if not s.get("areas") and s
    state["policy"] = policy_scan(extra)
    state["pli_news"] = pli_scan()
    state["bottlenecks"] = bottleneck_scan()
    state["watch"] = watch
    state["universe_n"] = len(by_sym)
    state["scan_note"] = scan_note
    state["full_at"] = now_ist().strftime("%d %b %Y %H:%M IST")
    save_raw(raw)
    return raw, state

def live_refresh():
    raw, state = load_raw(), load_state()
    if not raw:
        print("No saved data yet - running a full scan first.")
        return full_scan()
    todo = sorted([s for s in raw.values() if prio_of(s) >= 1],
```

```
                key=lambda s: (-prio_of(s), (s.get("m") or {}).get("ts", 0)))[:
    for s in todo:
        m = market_metrics(s["sym"])
        if m:
            m["ts"] = time.time()
            s["m"] = m
        s["news"] = get_news(s["name"]); time.sleep(0.5)
    save_raw(raw)
    return raw, state

def render(raw, state, alerts=True):
    stocks = list(raw.values())
    demo = bool(state.get("demo"))
    rs_list = sorted((s["m"]["rs_raw"], i) for i, s in enumerate(stocks) if s.get
    for rank_, (_, i_) in enumerate(rs_list):
        stocks[i_]["m"]["rs_rank"] = max(1, min(99, round(99 * rank_ / max(1, len
    state["market"] = state.get("market") if demo else market_overview()
    cyc = sector_cycle(stocks, state.get("policy"))
    views = [build_view(s, cyc) for s in stocks]
    for s, v in zip(stocks, views):
        s["score"] = v["score"]
    cyc = sector_cycle(stocks, state.get("policy"))
    views.sort(key=lambda v: (-{"Bullish": 2, "Watch": 1, "Avoid": 0}[v["verdict"
    featured = [v for v in views if v["tier"] != "-" or v["watch"] or v["verdict"

    news = []
    for v in featured:
        for n in v["news"]:
            news.append({**n, "sym": v["sym"], "name": v["name"]})
    news.sort(key=lambda n: n["age_h"])

    cust = {}
    for v in views:
        for c in v["customers"]:
            e = cust.setdefault(c["name"], {"name": c["name"], "region": c["regio
            e["cos"].append({"sym": v["sym"], "name": v["name"], "call": v["call"
    orders = sorted(cust.values(), key=lambda e: -len(e["cos"]))[:80]
    for e in orders:
        e["cos"] = sorted(e["cos"], key=lambda c: -c["n"])[:15]

    acc = [{"sym": v["sym"], "name": v["name"], "call": v["call"], "score": v["sc
            "signals": [x["t"] for x in v["holding"]["signals"]][:4]}
           for v in views if (v.get("holding") or {}).get("inst") == "Accumulatin
    acc = sorted(acc, key=lambda x: -x["score"])[:60]
    deals, prefs = [], []
    for v in views:
```

```
    for it in v["invest"]:
        row = {"sym": v["sym"], "name": v["name"], "title": it["title"], "lin
               "ago": it.get("ago", ""), "age_h": it.get("age_h", 0)}
        (prefs if it["kind"].startswith("Preferential") else deals).append(ro
    for fl in v["filings"]:
        if "preferential" in fl["text"].lower() or "warrant" in fl["text"].lo
            prefs.append({"sym": v["sym"], "name": v["name"], "title": fl["te
deals.sort(key=lambda x: x["age_h"]); prefs.sort(key=lambda x: x["age_h"])
brokers = []
for v in views:
    for b in v["broker"]:
        brokers.append({**b, "sym": v["sym"], "name": v["name"], "scall": v["
brokers.sort(key=lambda b: (not b.get("who"), b.get("age_h", 0)))

cnt_a, cnt_s = {}, {}
for v in views:
    for a_ in v["areas"]:
        cnt_a[a_] = cnt_a.get(a_, 0) + 1
    if not v["areas"] and v["sector"]:
        cnt_s[v["sector"]] = cnt_s.get(v["sector"], 0) + 1
areas = [{"area": a_, "n": cnt_a[a_], "kind": "area"} for a_ in AREAS if cnt_
areas += [{"area": k, "n": n, "kind": "sector"} for k, n in sorted(cnt_s.item

summary = [{"s": v["sym"], "n": v["name"], "a": v["areas"], "t": v["sector"]
            "d": v["day"], "v": v["call"], "sc": v["score"], "tr": v["tier"],
            "rv": (v["results"] or {}).get("verdict", ""), "tg": v["targets"]
            "sy": (v["results"] or {}).get("sales_yoy"), "py": (v["results"]
            "mg": (v["margin"] or {}).get("verdict", ""), "ad": v["adx"], "rs
            "mb": v["mb"]["score"], "mbc": v["mb"]["call"], "pu": v["mb"]["pu
            "ib": 1 if v["mb"]["insider_buy"] else 0, "pg": v["mb"]["p3"],
            "vr": round(v["vol_ratio"], 1) if v.get("vol_ratio") else None,
            "oc": v["ord"]["cr"], "op": v["ord"]["pct"], "on": v["ord"]["n"],
            "a4": 1 if v["ath4"] else 0, "aq": 1 if v["athq"] else 0, "ar": 1
            "v3": r1(v.get("vol_max3")), "tv": round(v["turnover_cr"], 2) if
            "r2": r1(v.get("ret2d")), "r5": r1(v.get("ret5d")), "cr": (v.get(
            "rp": (v["results"] or {}).get("period", ""), "qs": r1((v["result
            "pq": r1((v["results"] or {}).get("profit_qoq")), "qm": r1((v["re
            "afy": (v.get("annual") or {}).get("fy", ""), "ap": r1((v.get("an
            "app": r1((v.get("annual") or {}).get("prev_profit")), "ay": r1((
            "asy": r1((v.get("annual") or {}).get("sales_yoy")), "asl": r1((v
            "atp": r1((v.get("annual") or {}).get("ttm_profit")),
            "dd": round(v["dd"], 1) if v["dd"] is not None else None,
            "pn": round(v["pn"], 1) if v["pn"] is not None else None, "pa": r
            "fy": round(v["capex"]["fa_yoy"], 1) if v["capex"].get("fa_yoy")
            "pm": round(v["capex"]["pm_yoy"], 1) if v["capex"].get("pm_yoy")
            "cw": round(v["capex"]["cwip_pct"], 1) if v["capex"].get("cwip_pc
```

```
            "cf": round(v["capex"]["cfo_yoy"], 1) if v["capex"].get("cfo_yoy"
            "co": round(v["capex"]["cfo_now"], 1) if v["capex"].get("cfo_now"
            "ex": 1 if v["expanding"] else 0,
            "tc": v["tscore"], "tk": sorted({x["k"] for x in v["triggers"]}),
            "tl": [x["t"][:70] for x in sorted(v["triggers"], key=lambda x: -

btl = state.get("bottlenecks") or {}
if demo and not btl:
    btl = {b["id"]: {"news": [{"title": f"Demo headline: supply of {b['title'
                               "link": "https://example.com", "src": "Demo Wi
                     "alarm": 4, "heat": ["High", "Medium", "Low"][j % 3]} fo
bt_out = []
for b in BOTTLENECKS:
    d_ = btl.get(b["id"], {})
    ben = [v for v in views if (set(v["areas"]) & set(b["areas"])) or (set(v.
    ben.sort(key=lambda v: (v["tier"] == "-", -v["score"]))
    bt_out.append({"id": b["id"], "title": b["title"], "chain": b["chain"], "
                   "heat": d_.get("heat", "n/a"), "alarm": d_.get("alarm", 0)
                   "stocks": [{"s": v["sym"], "n": v["name"], "call": v["call
                               "rv": (v["results"] or {}).get("verdict", ""),
log = state.setdefault("log", {})
today_s = now_ist().strftime("%Y-%m-%d")
if demo and not log:
    rnd = random.Random(5)
    for j, v in enumerate([x for x in views if x["call"] == "BUY" and x["pric
        pr0 = v["price"] / rnd.uniform(0.82, 1.35)
        log[v["sym"]] = {"d": (now_ist() - timedelta(days=25 + j * 11)).strft
                         "call": "BUY", "mb": "MULTIBAGGER BUY" if j % 3 == 0
                         "stop": round(pr0 * 0.86, 1), "t1": round(pr0 * 1.2,
for v in views:
    if v["price"] and (v["call"] == "BUY" or v["mb"]["call"] == "MULTIBAGGER
        e = log.get(v["sym"])
        if not e:
            pl_ = v.get("plan") or {}
            log[v["sym"]] = {"d": today_s, "p": round(v["price"], 2), "n": v[
                             "score": v["score"], "stop": pl_.get("stop"), "t
        elif v["mb"]["call"] == "MULTIBAGGER BUY":
            e["mb"] = "MULTIBAGGER BUY"
vmap = {v["sym"]: v for v in views}
trows = []
for sym_, e in list(log.items())[-3000:]:
    v = vmap.get(sym_)
    if not v or not v["price"] or not e.get("p"):
        continue
    e["hi"] = max(e.get("hi") or 0, v["price"])
    trows.append({"s": sym_, "n": e["n"], "d": e["d"], "p": e["p"], "now": ro
```

```
                  "days": (datetime.strptime(today_s, "%Y-%m-%d") - datetime.
                  "stop": e.get("stop"), "t1": e.get("t1"), "stop_hit": bool(
                  "t1_hit": bool(e.get("t1") and e["hi"] >= e["t1"]), "call":
trows.sort(key=lambda r: r["d"], reverse=True)
track = {"n": len(trows), "win": round(100 * sum(r["ret"] > 0 for r in trows)
         "avg": round(sum(r["ret"] for r in trows) / len(trows), 1) if trows
         "stops": sum(r["stop_hit"] for r in trows), "t1": sum(r["t1_hit"] fo
quiet = sorted([{"sym": v["sym"], "name": v["name"], "call": v["call"], "scor
                "rv": (v["results"] or {}).get("verdict", ""), "flat": v["ac
                "adx": v["adx"], "rs": v["rs_rank"]} for v in views if v.get
out = {
    "updated": now_ist().strftime("%d %b %Y, %H:%M IST"), "full_at": state.ge
    "market": state.get("market", {}), "policy": policy_view(state.get("polic
    "pli": {"schemes": [{k: v_ for k, v_ in p_.items() if k != "kw"} for p_ i
            "stocks": [{"name": v["name"], "sym": v["sym"], "theme": v["theme
                       for v in featured if v["pli"]]},
    "coverage": {"loaded": len(views), "universe": state.get("universe_n"), "
    "bottlenecks": bt_out,
    "future": [{"name": n_, "drivers": AREAS[n_]["drivers"], "risks": AREAS[n
    "stocks": featured, "accum": quiet, "news": news[:60], "areas": areas, "o
    "big": {"acc": acc, "deals": deals[:60], "prefs": prefs[:60]}, "brokers":
    "focus_themes": [c["theme"] for c in cyc if c["stance"].startswith("FOCUS
    "avoid_themes": [c["theme"] for c in cyc if c["stance"] in ("AVOID", "RED
    "counts": {"candidates": len(views), "A": sum(v["tier"] == "A" for v in v
               "bullish": sum(v["verdict"] == "Bullish" for v in views)},
}
if demo:   # the offline demo page cannot fetch files, so embed everything
    out["univ"] = summary
    out["detail"] = {v["sym"]: v for v in views}

os.makedirs(f"{DATA_DIR}/d", exist_ok=True)
keep = set()
for v in views:
    fn = safe_sym(v["sym"]) + ".json"
    keep.add(fn)
    json.dump(clean(v), open(f"{DATA_DIR}/d/{fn}", "w"), separators=(",", ":"
for fn in os.listdir(f"{DATA_DIR}/d"):
    if fn not in keep:
        os.remove(f"{DATA_DIR}/d/{fn}")
json.dump(clean(summary), open(f"{DATA_DIR}/summary.json", "w"), separators=(
if alerts and not demo:
    do_alerts(views, state)
save_state(state)
payload = json.dumps(clean(out), ensure_ascii=True).replace("</", "<\\/")
open(f"{OUT_DIR}/index.html", "w", encoding="utf-8").write(HTML.replace("__DA
open(f"{OUT_DIR}/.nojekyll", "w").write("")
```

```
    print(f"Dashboard written to {OUT_DIR}/ : {len(views)} stocks, {len(featured)
          f"(A={out['counts']['A']} B={out['counts']['B']} buy={out['counts']['bu

# ---------------------------------------------------------------------------
# Demo data (to preview the dashboard offline)
# ---------------------------------------------------------------------------
def demo_data():
    random.seed(7)
    names = [("Apex Precision Forge", 0), ("AeroDyn Components", 0), ("Voltgrid T
             ("Powerlink Cables", 1), ("AquaPure Chemicals", 2), ("FluoroSpec Ind
             ("Sensora EMS", 3), ("RailTrack Systems", 4), ("AutoMatic Controls",
             ("PowerTech Insulators", 1), ("PureFlow Water", 2), ("Microfab Semic
    themes = ["Precision engineering & components", "Transmission, transformers &
              "Electronics & EMS", "Railways"]
    stocks = []
    for i, (n, ti) in enumerate(names):
        strong = i % 3 != 2
        flags = [strong, strong, strong or i % 2 == 0, strong and i % 4 != 1, str
        price = random.uniform(300, 4000)
        d200 = price * random.uniform(0.7, 0.95) if strong else price * random.un
        f = {"period": "Jun 2026", "q_sales": 320.5, "q_profit": 48.2, "sales_yoy
             "profit_yoy": random.uniform(10, 70), "opm_now": 19, "opm_yr": 19 +
             "roce": random.uniform(14, 32), "pe": random.uniform(25, 90), "mcap"
             "ath_q_sales": flags[0], "ath_q_profit": flags[1], "ath_y_sales": fl
             "profit_breakout": flags[4], "y_sales": 1200,
             "debtor_days": 62, "debtor_prev": 60 if strong else 48, "inv_days":
        m = {"price": price, "day": random.uniform(-2, 4), "dma50": price * rando
             "hi52": price * random.uniform(1.0, 1.15), "vol": 1, "avg20": 1, "vo
             "turnover_cr": random.uniform(0.8, 40), "ret1m": random.uniform(-5,
             "asof": "demo", "adx": random.uniform(14, 42), "pdi": 27, "mdi": 14,
             "rs_raw": random.uniform(-10, 80), "ret2d": random.uniform(-1.5, 3),
             "vol_max3": random.uniform(0.7, 2.6), "rs_nifty6m": random.uniform(-
             "dma_state": random.choice(["golden_cross", "converging_up", "squeez
        news = []
        if strong:
            news = [{"title": f"{n} bags Rs {random.randint(80, 900)} crore expor
                     "src": "Demo Wire", "ago": f"{random.randint(1, 20)}h ago",
        elif i % 3 == 2:
            news = [{"title": f"{n} faces SEBI probe over disclosures", "link": "
                     "ago": "5h ago", "age_h": 5, "s": -1}]
        stocks.append({"sym": n.split()[0].upper(), "name": n, "url": "https://ww
                       "theme": themes[ti], "f": f, "m": m, "news": news})
    for i, s in enumerate(stocks):
        if i % 3 != 2:
            s["concall"] = {"tone": "Positive", "ob_cr": 2400, "ob_text": "Order
                            "order_book": ["Our order book stands at Rs 2,400 cro
```

```
                        "guidance": ["We expect around 25 percent revenue gro
                        "capex": ["Phase 2 expansion of 40 percent more capac
                        "risks": ["Raw material price volatility remains a wa
        s["credit"] = {"rating": "AA/Stable", "agency": "CRISIL", "date": "22
        s["broker"] = [{"title": "Brokerage initiates Buy with target of Rs 1
                        "ago": "1d ago", "age_h": 24, "s": 1, "view": 1}]
        if i % 2 == 0:
            s["pli"] = [{"title": f"{s['name']} gets PLI approval, incentive
                         "ago": "2d ago", "age_h": 48, "s": 1, "amt": "Rs 320
ql = ["Sep 2024", "Dec 2024", "Mar 2025", "Jun 2025", "Sep 2025", "Dec 2025",
for i, s in enumerate(stocks):
    base, gr = random.uniform(100, 400), random.uniform(-0.03, 0.12)
    sales = [round(base * (1 + gr) ** k, 1) for k in range(8)]
    prof = [round(x * random.uniform(0.08, 0.16), 1) for x in sales]
    s["f"]["results"] = results_view(ql, {"sales": sales, "net profit": prof,
                                          "opm %": [round(14 + k * random.uni
    ex = [round(x * (0.84 - (k * 0.01 if i % 3 != 2 else -k * 0.01)), 1) for
    s["f"]["margin"] = margin_view({"sales": sales, "expenses": ex, "operatin
                                    "opm %": [round(100 * (a - b) / a, 1) for
    s["f"]["cogs"] = {"now": 52, "yr": 56, "q": 53} if i % 2 == 0 else None
    if i in (0, 3):
        s["f"]["results"]["verdict"] = "Excellent"
        s["f"]["results"].update(profit_yoy=95.0 if i == 0 else 78.0, sales_y
        if i == 3:
            s["m"]["hi52"] = s["m"]["price"] / 0.62
        s["m"].update(ret2d=0.4, ret5d=1.1, ret1m=3.0, vol_ratio=1.9, vol_max
    up = 1 if i % 3 != 2 else -1
    s["f"]["cagr"] = {"sales3": random.uniform(8, 35), "sales5": 15, "profit3
    if i % 3 == 0:
        s["insider"] = [{"title": f"Promoter buys 1.5 lakh shares of {s['name
                         "src": "Demo Wire", "ago": "9d ago", "age_h": 216, "
    s["holding"] = holding_view(ql, {
        "promoters": [round(55 + (0.15 if i % 2 == 0 else -0.1) * k, 1) for k
        "fiis": [round(4 + up * k * 0.4, 1) for k in range(8)],
        "diis": [round(0.5 + up * k * 0.5 + (3 if up > 0 else 6), 1) for k in
        "government": [0.1] * 8, "public": [round(40 - up * k * 0.3, 1) for k
        "no. of shareholders": [20000 + k * (2500 if i % 4 == 0 else 200) for
    if i % 2 == 0:
        s["invest"] = [{"title": f"Mutual fund buys 1.2% stake in {s['name']}
                        "src": "Demo Wire", "ago": "6d ago", "age_h": 144, "s
        if i % 4 == 0:
            s["invest"].append({"title": f"{s['name']} board approves prefere
                                "src": "Demo Wire", "ago": "20d ago", "age_h"
    s["docs"] = {"filings": [{"text": "Allotment of equity shares on preferen
for i, s in enumerate(stocks):
    s["areas"] = [s["theme"]] + (["Nuclear"] if i % 5 == 0 else []) + (["Data
```

```
        s["sector"] = "Capital Goods"
        s["about"] = "Demo company making precision components for aerospace, nuc
        s["m"]["ts"] = time.time()
        if s.get("concall"):
            s["concall"]["targets"] = [{"area": "Aerospace", "n": 9}, {"area": "N
            s["concall"]["focus"] = ["We are targeting new customers in the data
            s["concall"]["customers"] = [{"name": "Vertiv", "region": "Global", "
                                         "ctx": "We received a repeat order from
        s["orders"] = ([{"title": f"{s['name']} bags Rs 320 crore order from Safr
                        "ago": "5d ago", "age_h": 120, "cust": ["Safran"], "amt_
        if i % 2 == 0:
            s["f"]["mcap"] = 1200 + i * 90
        if i in (4, 6, 9):
            s["f"]["mcap"] = 300 + i * 20
        s["areas"] = list(dict.fromkeys((s.get("areas") or []) + [FUTURE_AREAS[i
        s["f"]["cfo_ratio"] = random.choice([0.95, 1.1, 0.3])
        s["f"]["capex"] = {"fa_yoy": random.uniform(-5, 45), "pm_yoy": random.uni
                           "cfo_now": random.uniform(40, 300), "cfo_yoy": random.
        pp_, gr_ = random.uniform(60, 120), random.uniform(-10, 90)
        s["f"]["annual"] = {"fy": "Mar 2026", "sales": 1200, "profit": round(pp_
                            "prev_profit": round(pp_, 1), "profit_yoy": gr_, "sal
                            "ttm_profit": round(pp_ * 1.1 * (1 + gr_ / 100), 1),
        s["f"]["de"] = random.choice([0.1, 0.4, 1.8])
        if s.get("concall"):
            s["concall"].update(certs=["AS9100", "NADCAP"] if i % 2 == 0 else [],
                                top_cust_pct=45 if i == 3 else None, v=3)
        for b in s.get("broker", []):
            b.update(who=["Kotak"], target=1500, call="Buy")
    pol = {"themes": {}, "newsletters": [{"title": "Demo newsletter: why T&D cape
           "src": "demo-letter", "ago": "6h ago", "age_h": 6, "s": 1, "theme": th
    for i, th in enumerate(themes):
        its = [{"title": f"Cabinet approves Rs {1000 * (i + 2)} crore scheme for
                "src": "PIB", "ago": f"{i + 1}d ago", "age_h": 24 * (i + 1), "s":
        pol["themes"][th] = {"items": its, "score": [12, 8, 5, 11, 2][i], "level"
    pli_news = [{"title": "Govt approves Rs 1,200 crore PLI incentive for 12 elec
                 "src": "Demo Wire", "ago": "3d ago", "age_h": 72, "amt": "Rs 1,2
    return {"stocks": stocks, "demo": True, "policy": pol, "pli_news": pli_news,
            "market": {"nifty": {"price": 24500, "ret1m": 2.1}, "vix": {"price":
                       "mood": "Constructive - Nifty above 200-DMA and rising. Br

# ---------------------------------------------------------------------------
# Dashboard HTML (single file, mobile-first)
# ---------------------------------------------------------------------------
HTML = r"""<!doctype html><html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1,viewport-fit=co
<title>India Moat Screener</title>
```

```
<style>
:root{--bg:#f4f5f7;--card:#fff;--tx:#141821;--mut:#667085;--bd:#e3e6ec;--g:#0a8f4
@media(prefers-color-scheme:dark){:root{--bg:#0d1015;--card:#161a21;--tx:#edf0f5;
*{box-sizing:border-box}body{margin:0;background:var(--bg);color:var(--tx);font:1
header{padding:calc(12px + env(safe-area-inset-top,0px)) 14px 8px}
h1{font-size:19px;margin:0}.sub{color:var(--mut);font-size:12px}
.mood{margin:8px 0 0;padding:9px 11px;border-radius:10px;font-size:13px}
.mood.g{background:var(--gb);color:var(--g)}.mood.a{background:var(--ab);color:va
nav{position:sticky;top:0;background:var(--bg);display:flex;gap:6px;overflow-x:au
nav button{flex:none;border:1px solid var(--bd);background:var(--card);color:var(
nav button.on{background:var(--tx);color:var(--bg);border-color:var(--tx)}
main{padding:12px 14px;max-width:760px;margin:auto}
.stats{display:grid;grid-template-columns:repeat(4,1fr);gap:8px;margin-bottom:12p
.stat{background:var(--card);border:1px solid var(--bd);border-radius:12px;paddin
.stat b{display:block;font-size:20px}.stat span{font-size:11px;color:var(--mut)}
.card{background:var(--card);border:1px solid var(--bd);border-radius:14px;paddin
.row{display:flex;justify-content:space-between;gap:8px;align-items:flex-start}
.name{font-weight:700;font-size:16px}.theme{font-size:12px;color:var(--mut)}
.badge{display:inline-block;padding:2px 9px;border-radius:12px;font-size:12px;fon
.badge.g{background:var(--gb);color:var(--g)}.badge.a{background:var(--ab);color:
.score{font-size:24px;font-weight:800;text-align:right;line-height:1}.score small
.g{color:var(--g)}.r{color:var(--r)}.a{color:var(--a)}
.kv{display:flex;flex-wrap:wrap;gap:6px 14px;margin:8px 0;font-size:13px}.kv span
ul{margin:4px 0 6px 18px;padding:0;font-size:13.5px}li{margin:2px 0}
.sec{font-size:12px;font-weight:700;margin-top:8px;text-transform:uppercase;lette
.plan{background:var(--bg);border-radius:10px;padding:9px;font-size:13px;margin-t
.links a,.nw{color:var(--tx)}.links{display:flex;gap:8px;margin-top:9px}
.links a{font-size:12px;border:1px solid var(--bd);border-radius:8px;padding:5px
.nw{display:block;font-size:13px;text-decoration:none;padding:6px 0;border-top:1p
.nw small{color:var(--mut)}.nw.p{border-left:3px solid var(--g);padding-left:8px}
details summary{cursor:pointer;color:var(--mut);font-size:13px;margin-top:6px}
.bar{height:6px;background:var(--bd);border-radius:4px;overflow:hidden;margin:3px
h2{font-size:15px;margin:16px 0 8px}.empty{color:var(--mut);padding:20px;text-ali
input,select{width:100%;padding:9px;border-radius:10px;border:1px solid var(--bd)
.ht{border-collapse:collapse;font-size:12px;width:100%}.ht td,.ht th{padding:4px
.chip{display:inline-block;font-size:11px;padding:1px 7px;border-radius:10px;back
.badge.b{background:#e0ecff;color:#1d4ed8}@media(prefers-color-scheme:dark){.badg
.segs{display:flex;gap:6px;overflow-x:auto;margin:8px 0}.segs button{flex:none;bo
.more{width:100%;padding:10px;border-radius:10px;border:1px solid var(--bd);backg
.disc{color:var(--mut);font-size:11.5px;margin-top:20px}
</style></head><body>
<header><h1>India Moat Screener</h1><div class="sub" id="upd"></div><div id="mood
<nav id="nav"></nav><main id="app"></main>
<script>
const D=__DATA__;
const esc=t=>String(t==null?'':t).replace(/[&<>"']/g,c=>({'&':'&amp;','<':'&lt;',
```

```
const sg=(x,d=1)=>x==null?'-':(x>0?'+':'')+Number(x).toFixed(d)+'%';
const vcls=v=>v==='Bullish'?'g':v==='Watch'?'a':'r';
const tabs=[['picks','Rule screen'],['best','Best picks / compare'],['buy','Big b
let tab='picks',q='',th='';
document.getElementById('upd').textContent='Updated '+D.updated+(D.full_at?'  |
const M=D.market||{};
if(M.mood){const n=M.nifty,v=M.vix;document.getElementById('mood').innerHTML='<di
function nav(){document.getElementById('nav').innerHTML=tabs.map(t=>'<button clas
function go(t){tab=t;nav();draw();scrollTo(0,0)}
function card(s){
 const vc=s.call?ccls(s.call):vcls(s.verdict),p=s.plan||{};
 const why=s.why.slice(0,3).map(x=>'<li>'+esc(x)+'</li>').join(''),whyMore=s.why.
 const risk=s.risks.map(x=>'<li>'+esc(x)+'</li>').join('');
 const nw=s.news.slice(0,3).map(n=>'<a class="nw '+(n.s>0?'p':n.s<0?'n':'')+'" hr
 return '<div class="card"><div class="row"><div><div class="name">'+esc(s.name)+
 '<div><div class="score '+vc+'">'+s.score+'<small>/100</small></div></div></div>
 '<div style="margin-top:6px"><span class="badge '+vc+'">'+esc(s.call||s.verdict)
 '<div class="kv"><div><span>Price</span> '+(s.price?'Rs '+s.price.toFixed(1):'-'
 '<div><span>Volume</span> <b class="'+(s.vol_ratio>=1.5?'g':'')+'">'+(s.vol_rati
 '<div><span>Liquidity</span> '+(s.turnover_cr!=null?'Rs '+s.turnover_cr.toFixed(
 '<div><span>P/E</span> '+(s.pe?s.pe.toFixed(0):'-')+'</div><div><span>ROCE</span
 (why?'<div class="sec g">Why bullish</div><ul>'+why+'</ul>':'')+
 (whyMore?'<details><summary>More reasons</summary><ul>'+whyMore+'</ul></details>
 (risk?'<div class="sec r">Watch out</div><ul>'+risk+'</ul>':'')+
 (p.stop?'<div class="plan"><b>Plan:</b> '+esc(p.entry)+'<br><b>Stop-loss:</b> Rs
 (nw?'<details><summary>News ('+s.news.length+')</summary>'+nw+'</details>':'')+d
 '<div class="links"><a href="'+esc(s.links.screener)+'" target="_blank" rel="noo
}
function picks(){
 const c=D.counts;let h='<div class="stats"><div class="stat"><b class="g">'+c.bu
 const bull=D.stocks.filter(s=>s.verdict==='Bullish'),watch=D.stocks.filter(s=>s.
 h+='<div class="card"><div class="sec g">Focus sectors now</div><div style="font
 if(watch.length)h+='<h2>Watchlist - wait for a trigger</h2>'+watch.map(card).joi
 return h;}
function sectors(){return D.sectors.map(c=>{
 return '<div class="card"><div class="row"><div class="name">'+esc(c.theme)+'</d
 '<div class="'+c.tone+'" style="margin:4px 0 8px;font-weight:600">'+esc(c.phase)
 '<div class="theme">Government policy support: <b class="'+(c.policy_level==='Hi
 (c.pli&&c.pli.length?'<div class="sec g">Govt incentive (PLI) support</div>'+c.p
 '<div class="theme">Stocks at record quarterly profit: '+c.earn_ath_pct+'%</div>
 '<div class="theme">Above 200-DMA: '+c.above200_pct+'%</div><div class="bar"><i
 '<div class="theme">Margins expanding: '+c.opm_up_pct+'%</div><div class="bar"><
 '<div class="kv"><div><span>Median 1m</span> '+sg(c.ret1m)+'</div><div><span>Med
 (c.drivers?'<div class="sec g">Structural drivers</div><div style="font-size:13.
 (c.risks?'<div class="sec r">Risks</div><div style="font-size:13.5px">'+esc(c.ri
 '<div class="theme" style="margin-top:8px">Top ranked: '+esc(c.top.join(', '))+'
```

```
function focus(){
 const f=D.sectors.filter(c=>c.stance.startsWith('FOCUS')),a=D.sectors.filter(c=>
 const sc=(c)=>'<div class="card"><div class="row"><b>'+esc(c.theme)+'</b><span c
 let h='<h2 class="g">Focus on</h2>'+(f.length?f.map(sc).join(''):'<div class="em
 h+='<h2 class="a">Wait</h2>'+(w.length?w.map(sc).join(''):'<div class="empty">-<
 h+='<h2 class="r">Avoid (sectors)</h2>'+(a.length?a.map(sc).join(''):'<div class
 const av=D.stocks.filter(s=>s.tier!=='-'&&s.verdict==='Avoid').slice(0,12);
 h+='<h2 class="r">Stocks with strong numbers but a red flag</h2>'+(av.length?av.
 h+='<h2>Simple rules</h2><div class="card"><ul><li>Prefer Tier A/B stocks in sec
 return h;}
function newsTab(){return D.news.length?D.news.map(n=>'<div class="card" style="p
function policyTab(){
 const P=D.policy||{themes:[],newsletters:[]};
 const it=n=>'<a class="nw '+(n.s>0?'p':n.s<0?'n':'')+'" href="'+esc(n.link)+'" t
 let h='<div class="card"><div class="theme">Government and policy headlines from
 h+=P.themes.map(t=>'<div class="card"><div class="row"><div class="name">'+esc(t
 h+='<h2>Newsletters you follow</h2>'+(P.newsletters.length?P.newsletters.map(n=>
 return h;}
const wcv=(a,b)=>a==null||b==null?'-':'<b class="'+(b>a*1.03+0.5?'r':'g')+'">'+Ma
const ul=a=>a&&a.length?'<ul>'+a.map(x=>'<li>'+esc(x)+'</li>').join('')+'</ul>':'
const lk=(u,t)=>'<a href="'+esc(u)+'" target="_blank" rel="noopener">'+t+'</a>';
const selH=(opts,cur,fn)=>'<select onchange="'+fn+'">'+opts.map(o=>'<option value
const CALLS=[['NOAVOID','Call: hide AVOID'],['BUYHOLD','Call: BUY and HOLD'],['BU
const RECS=[['A4','Records: quarterly and annual sales and profit'],['Q','Records
const callOkG=(c,u)=>c==='ALL'||(c==='NOAVOID'?u.v!=='AVOID':c==='BUYHOLD'?(u.v==
const athOkG=(m,u)=>m==='A4'?!!u.a4:m==='Q'?!!u.aq:m==='AY'?!!u.ar:true;
const crf=n=>n==null?'-':(Math.abs(n)<100?n.toFixed(1):Math.round(n).toLocaleStri
const athTag=u=>u.a4?'<span class="chip g">All 4 records</span>':u.aq?'<span clas
function recRow(u,mode){
 const x=(u.pn!=null&&u.pa>0)?u.pn/u.pa:null;
 let kv='<div><span>Quarter profit</span> <b>'+(u.pa!=null&&u.pn!=null?'Rs '+crf(
 if(mode==='disc')kv+='<div><span>Below 52-week high</span> <b class="a">'+(u.dd=
 if(mode==='exp')kv+='<div><span>Fixed assets</span> <b>'+pc(u.fy)+'</b></div><di
 kv+='<div><span>Quarter</span> '+esc(u.rv||'-')+'</div><div><span>Score</span> '
 return '<div class="card" style="padding:10px;cursor:pointer" onclick="openP(\''
const capOkG=(i,u)=>{if(i==='ALL')return true;const b=(D.caps||[])[+i];return !!b
const capSelH=(cur,fn)=>selH([['ALL','All company sizes']].concat((D.caps||[]).ma
const ratChip=u=>u.cr?'<span class="chip">Credit rating '+esc(u.cr)+'</span>':'';
const moreBtn=(n,total,fn)=>total>n?'<button class="more" onclick="'+fn+'">Show 4
let buyN=40,buyVol='2',buyMove='up',buyLiq='0.5',buyRec='ANY',buyCall='NOAVOID',b
function buyTab(){return '<div class="card"><div class="theme">Stocks with a big
function buyRow(u){
 const vm=Math.max(u.vr||0,u.v3||0);
 return '<div class="card" style="padding:10px;cursor:pointer" onclick="openP(\''
function renderBuy(){
 const el=document.getElementById('buybox');if(!el)return;
```

```
 if(!U){el.innerHTML='<div class="empty">Loading all stocks...</div>';loadUniv().
 const vm=u=>Math.max(u.vr||0,u.v3||0);
 const rows=U.filter(u=>vm(u)>=+buyVol&&(buyMove==='any'||(buyMove==='up'?(u.d||0
 const key={vol:vm,val:u=>vm(u)*(u.tv||0),move:u=>u.d||0,sc:u=>u.sc}[buySort]||vm
 rows.sort((a,b)=>key(b)-key(a));
 el.innerHTML=selH([['1.5','Volume: 1.5x average or more'],['2','Volume: 2x avera
  selH([['up','Price: rising today'],['up2','Price: up 2% or more today'],['any',
  selH([['0.25','Liquidity: Rs 0.25 Cr a day or more'],['0.5','Liquidity: Rs 0.5
  selH([['ANY','Records: not required'],['Q','Records: quarterly sales and profit
  selH(CALLS,buyCall,"buyCall=this.value;buyN=40;renderBuy()")+capSelH(buyCap,"bu
  selH([['vol','Sort: volume multiple'],['val','Sort: money traded'],['move','Sor
  '<h2>Big buying volume ('+rows.length+')</h2>'+(rows.length?rows.slice(0,buyN).
let qpN=40,qpRec='Q',qpCall='NOAVOID',qpCap='ALL',qpSort='py',qpMin='0';
function qpTab(){return '<div class="card"><div class="theme">Latest quarterly pr
function qpRow(u){
 const x=(u.pn!=null&&u.pa>0)?u.pn/u.pa:null;
 return '<div class="card" style="padding:10px;cursor:pointer" onclick="openP(\''
function renderQp(){
 const el=document.getElementById('qpbox');if(!el)return;
 if(!U){el.innerHTML='<div class="empty">Loading all stocks...</div>';loadUniv().
 const rows=U.filter(u=>u.pn!=null&&athOkG(qpRec,u)&&callOkG(qpCall,u)&&capOkG(qp
 const key={py:u=>u.py==null?-9999:u.py,pn:u=>u.pn==null?-9999:u.pn,sy:u=>u.sy==n
 rows.sort((a,b)=>key(b)-key(a));
 el.innerHTML=selH([['Q','Records: quarterly sales and profit at all-time high'],
  selH([['0','Profit growth vs year ago: 0% or more'],['25','Profit growth: +25%
  selH(CALLS,qpCall,"qpCall=this.value;qpN=40;renderQp()")+capSelH(qpCap,"qpCap=t
  selH([['py','Sort: profit growth vs year ago'],['pn','Sort: profit amount'],['s
  '<h2>Quarterly profit ('+rows.length+')</h2>'+(rows.length?rows.slice(0,qpN).ma
let ypN=40,ypRec='AY',ypCall='NOAVOID',ypCap='ALL',ypSort='ay',ypMin='0';
function ypTab(){return '<div class="card"><div class="theme">Full-year profit of
function ypRow(u){
 const x=(u.ap!=null&&u.app>0)?u.ap/u.app:null;
 return '<div class="card" style="padding:10px;cursor:pointer" onclick="openP(\''
function renderYp(){
 const el=document.getElementById('ypbox');if(!el)return;
 if(!U){el.innerHTML='<div class="empty">Loading all stocks...</div>';loadUniv().
 const rows=U.filter(u=>u.ap!=null&&athOkG(ypRec,u)&&callOkG(ypCall,u)&&capOkG(yp
 const key={ay:u=>u.ay==null?-9999:u.ay,ap:u=>u.ap==null?-9999:u.ap,asy:u=>u.asy=
 rows.sort((a,b)=>key(b)-key(a));
 el.innerHTML=selH([['AY','Records: annual sales and profit at all-time high'],['
  selH([['0','Profit growth vs last year: 0% or more'],['25','Profit growth: +25%
  selH(CALLS,ypCall,"ypCall=this.value;ypN=40;renderYp()")+capSelH(ypCap,"ypCap=t
  selH([['ay','Sort: profit growth vs last year'],['ap','Sort: profit amount'],['
  '<h2>Annual profit, year on year ('+rows.length+')</h2>'+(rows.length?rows.slic
let rcPy='70',rcAth='A4',rcCall='NOAVOID';
function dblTab(){return '<div class="card"><div class="theme">Companies at their
```

```
function renderRec(){
 const el=document.getElementById('recbox');if(!el)return;
 if(!U){el.innerHTML='<div class="empty">Loading all stocks...</div>';loadUniv().
 const rows=U.filter(u=>athOkG(rcAth,u)&&u.py!=null&&u.py>=+rcPy&&callOkG(rcCall,
 el.innerHTML=selH([['50','Profit growth: +50% or more (1.5x)'],['70','Profit gro
  '<h2>Record sales and profit, profit about doubled ('+rows.length+')</h2>'+(row
let dsBand='30-50',dsPy='70',dsAth='A4',dsCall='NOAVOID';
function discTab(){return '<div class="card"><div class="theme">The same rule as
function renderDisc(){
 const el=document.getElementById('discbox');if(!el)return;
 if(!U){el.innerHTML='<div class="empty">Loading all stocks...</div>';loadUniv().
 const bd=dsBand.split('-').map(Number),lo=bd[0],hi=bd[1];
 const rows=U.filter(u=>athOkG(dsAth,u)&&u.py!=null&&u.py>=+dsPy&&u.dd!=null&&u.d
 el.innerHTML=selH([['20-30','Discount: 20 to 30% below 52-week high'],['30-50','
  '<h2>Records at a discount ('+rows.length+')</h2>'+(rows.length?rows.slice(0,60
let exFa='15',exAth='Q',exCall='NOAVOID';
function expTab(){return '<div class="card"><div class="theme">Companies with rec
function renderExp(){
 const el=document.getElementById('expbox');if(!el)return;
 if(!U){el.innerHTML='<div class="empty">Loading all stocks...</div>';loadUniv().
 const rows=U.filter(u=>athOkG(exAth,u)&&u.fy!=null&&u.fy>=+exFa&&(u.co==null||u.
 el.innerHTML=selH([['10','Fixed assets growth: +10% or more'],['15','Fixed asset
  '<h2>Record results with expanding capacity ('+rows.length+')</h2>'+(rows.lengt
function btlTab(){
 const B=D.bottlenecks||[],hc=h=>h==='High'?'g':h==='Medium'?'a':'b';
 let h='<div class="card"><div class="theme">Where the world is short of somethin
 return h+B.map(b=>'<div class="card"><div class="row"><div class="name">'+esc(b.
  '<div class="sec">The chain</div><div style="font-size:13.5px">'+esc(b.chain)+'
  '<div class="sec g">Why it is a bottleneck</div><div style="font-size:13.5px">'
  '<div class="sec">What to look for in the companies</div><div style="font-size:
  '<details><summary>Latest headlines ('+(b.news||[]).length+')</summary>'+((b.ne
  '<div class="sec">Indian stocks in this area ('+b.n+' loaded, best first)</div>
let fuIdx=0,fuRule='3',fuCall='NOAVOID',fuSort='mb';
function futureTab(){return '<div class="card"><div class="theme">Companies in te
function renderFuture(){
 const el=document.getElementById('fubox');if(!el)return;
 if(!U){el.innerHTML='<div class="empty">Loading all stocks...</div>';loadUniv().
 const F=D.future||[];if(!F.length){el.innerHTML='<div class="empty">No areas def
 const sel=(v,c)=>v==c?' selected':'';
 const cur=F[fuIdx]||F[0];
 const pass=(u,k)=>k==='ALL'||(k==='5'?u.tr==='A':k==='4'?(u.tr==='A'||u.tr==='B'
 const callOk=u=>fuCall==='ALL'||(fuCall==='NOAVOID'?u.v!=='AVOID':fuCall==='BUYH
 const all=U.filter(u=>(u.a||[]).indexOf(cur.name)>=0);
 const rows=all.filter(u=>pass(u,fuRule)&&callOk(u));
 const key={mb:u=>u.mb||0,sc:u=>u.sc,py:u=>u.py==null?-999:u.py,rs:u=>u.rs==null?
 rows.sort((a,b)=>key(b)-key(a));
```

```
 let h='<div class="segs">'+F.map((a,i)=>'<button class="'+(a===cur?'on':'')+'" o
 h+='<div class="card" style="padding:10px"><div class="sec g">Why this area</div
 h+='<div class="stats"><div class="stat"><b>'+all.length+'</b><span>Stocks</span
 h+='<select onchange="fuRule=this.value;renderFuture()"><option value="5"'+sel(f
 h+='<select onchange="fuCall=this.value;renderFuture()"><option value="NOAVOID"'
 h+='<select onchange="fuSort=this.value;renderFuture()"><option value="mb"'+sel(
 h+='<h2>'+esc(cur.name)+' ('+rows.length+')</h2>'+(rows.length?rows.slice(0,60).
 el.innerHTML=h;}
let trKind='ALL',trCall='NOAVOID',trCap='ALL';
const TRG={ALL:null,ORD:['order'],INS:['insider'],INST:['inst','deal','broker'],B
const TRN={ALL:'All triggers',ORD:'New orders',INS:'Insiders and promoters',INST:
function trigTab(){return '<div class="card"><div class="theme">Fresh reasons a s
function renderTrig(){
 const el=document.getElementById('trbox');if(!el)return;
 if(!U){el.innerHTML='<div class="empty">Loading all stocks...</div>';loadUniv().
 const sel=(v,c)=>v==c?' selected':'',B=D.caps||[];
 const capOk=u=>{if(trCap==='ALL')return true;const b=B[+trCap];return b&&u.c!=nu
 const callOk=u=>trCall==='ALL'||(trCall==='NOAVOID'?u.v!=='AVOID':trCall==='BUYH
 const kindOk=u=>!TRG[trKind]||(u.tk||[]).some(k=>TRG[trKind].indexOf(k)>=0);
 const rows=U.filter(u=>(u.tc||0)>0&&kindOk(u)&&callOk(u)&&capOk(u)).sort((a,b)=>
 let h='<div class="segs">'+Object.keys(TRG).map(k=>'<button class="'+(k===trKind
 h+='<select onchange="trCall=this.value;renderTrig()"><option value="NOAVOID"'+s
 h+='<select onchange="trCap=this.value;renderTrig()"><option value="ALL"'+sel(tr
 h+='<h2>'+TRN[trKind]+' ('+rows.length+')</h2>'+(rows.length?rows.slice(0,60).ma
 el.innerHTML=h;}
function trackTab(){
 const T=D.track||{n:0,rows:[]};
 let h='<div class="card"><div class="theme">Every stock that gets a BUY or MULTI
 if(!T.n)return h+'<div class="empty">Nothing tracked yet. Entries appear after t
 h+='<div class="stats"><div class="stat"><b>'+T.n+'</b><span>Calls tracked</span
 h+='<div class="theme" style="margin-bottom:8px">First target reached: '+T.t1+'
 return h+T.rows.map(r=>'<div class="card" style="padding:10px;cursor:pointer" on
let capSeg=0,capRule='3',capCall='BUYHOLD',capSort='sc';
function capTab(){return '<div class="card"><div class="theme">Choose a company s
const capRow=u=>'<div class="card" style="padding:10px;cursor:pointer" onclick="o
function renderCap(){
 const el=document.getElementById('capbox');if(!el)return;
 if(!U){el.innerHTML='<div class="empty">Loading all stocks...</div>';loadUniv().
 const B=D.caps||[],sel=(v,c)=>v==c?' selected':'';
 const inSeg=(u,b)=>u.c!=null&&u.c>=b.lo&&(b.hi==null||u.c<b.hi);
 const pass=(u,k)=>k==='ALL'||(k==='5'?u.tr==='A':k==='4'?(u.tr==='A'||u.tr==='B'
 const segs=B.map((b,i)=>{const all=U.filter(u=>inSeg(u,b));return {b:b,i:i,n:all
 const cur=segs[capSeg]||segs[0];
 if(!cur){el.innerHTML='<div class="empty">No data yet.</div>';return;}
 const rows=U.filter(u=>inSeg(u,cur.b)&&pass(u,capRule)&&(capCall==='ALL'||(capCa
 const key={sc:u=>u.sc,py:u=>u.py==null?-999:u.py,rs:u=>u.rs==null?-1:u.rs,vr:u=>
```

```
 rows.sort((a,b)=>key(b)-key(a));
 let h='<div class="segs">'+segs.map(s=>'<button class="'+(s.i===capSeg?'on':'')+
 h+='<div class="stats" style="grid-template-columns:repeat(3,1fr)"><div class="s
 h+='<select onchange="capRule=this.value;renderCap()"><option value="5"'+sel(cap
 h+='<select onchange="capCall=this.value;renderCap()"><option value="BUYHOLD"'+s
 h+='<select onchange="capSort=this.value;renderCap()"><option value="sc"'+sel(ca
 h+='<h2>'+esc(cur.b.name)+' stocks ('+rows.length+')</h2>'+(rows.length?rows.sli
 el.innerHTML=h;}
function ordBlock(s){const o=s.ord,c=s.concall||{};let h='';
 if(o&&(o.cr||(o.cust||[]).length||(o.files||[]).length)){h+='<div class="sec">Or
 if((c.certs||[]).length||(c.quals||[]).length||c.top_cust_pct!=null){h+='<div cl
 return h?'<details><summary>Orders, approvals, customer ties</summary>'+h+'</det
let owCap='SMALL';
function owTab(){const sel=(v,c)=>v==c?' selected':'';return '<div class="card"><
function renderOw(){
 const el=document.getElementById('owbox');if(!el)return;
 if(!U){el.innerHTML='<div class="empty">Loading all stocks...</div>';loadUniv().
 const lim=owCap==='MICRO'?500:owCap==='SMALL'?5000:1e12,okc=u=>u.c==null||u.c<li
 const big=U.filter(u=>okc(u)&&u.oc&&u.op!=null).sort((a,b)=>b.op-a.op);
 const named=U.filter(u=>okc(u)&&(u.ou||[]).length&&!(u.oc&&u.op!=null)).sort((a,
 const row=u=>'<div class="card" style="padding:10px;cursor:pointer" onclick="ope
 el.innerHTML='<div class="stats" style="grid-template-columns:repeat(2,1fr)"><di
  '<h2 class="g">Orders big compared with company size ('+big.length+')</h2>'+(bi
  '<h2>Customers named, order size not disclosed ('+named.length+')</h2>'+(named.
function deep(s){
 const c=s.concall||{},cr=s.credit||{},bk=s.broker||[],pl=s.pli||[];let h='';
 if(s.ob_text||(c.order_book&&c.order_book.length))h+='<div class="sec">Order boo
 if(c.tone){h+='<div class="sec">Concall: management tone '+esc(c.tone)+'</div>';
  if(c.guidance&&c.guidance.length)h+='<div class="theme">Outlook / guidance</div
  if(c.capex&&c.capex.length)h+='<div class="theme">Capacity / capex</div>'+ul(c.
  if(c.risks&&c.risks.length)h+='<div class="theme">Challenges mentioned</div>'+u
  if(c.link)h+='<div class="theme">'+lk(c.link,'Open transcript')+(c.ppt?' | '+lk
 if(cr.rating||cr.link)h+='<div class="sec">Credit rating</div><div style="font-s
 if(bk.length)h+='<div class="sec">Brokerage / analyst headlines</div>'+bk.map(n=
 if(pl.length)h+='<div class="sec">PLI scheme link</div>'+pl.map(n=>'<a class="nw
 return h?'<details><summary>Order book, concall, rating, PLI</summary>'+h+'</det
function pliTab(){
 const P=D.pli||{schemes:[],news:[],stocks:[]};
 const nw=n=>'<a class="nw p" href="'+esc(n.link)+'" target="_blank" rel="noopene
 let h='<div class="card"><div class="theme">PLI and similar schemes: how much th
 h+='<h2>Latest PLI news: who and how much</h2>'+(P.news.length?'<div class="card
 h+='<h2>Your scanned stocks with PLI headlines</h2>'+(P.stocks.length?P.stocks.m
 h+='<h2>Schemes</h2>'+P.schemes.map(p=>'<div class="card"><div class="row"><div
 return h;}
function dmaTab(){
 const lab={golden_cross:'Golden cross: 50-DMA just crossed above 200-DMA',conver
```

```
 const z=D.stocks.filter(s=>s.dma_state&&s.dma_state!=='death_cross');
 const good=z.filter(s=>s.tier!=='-'&&(s.wc||{}).wc_ok!==false).slice(0,20),rest=
 const item=s=>'<div class="theme" style="margin:8px 2px 4px"><b class="a">'+esc(
 let h='<div class="card"><div class="theme">Stocks where the 50-day and 200-day
 h+='<h2>Meeting point + strong fundamentals + healthy working capital</h2>'+(goo
 h+='<h2>Meeting point, but fundamentals or working capital not clean</h2>'+(rest
 return h;}
const rcol=v=>v==='Excellent'||v==='Good'?'g':v==='Mixed'?'a':'r';
const pc=x=>x==null?'-':(x>0?'+':'')+Math.round(x)+'%';
function rline(s){const r=s.results;if(!r)return '';return '<div class="theme" st
function mline(s){const m=s.margin;if(!m)return '';const c=s.cogs;return '<div cl
function accumTab(){
 const R=D.accum||[];
 return '<div class="card"><div class="theme">Results are good (Excellent or Good
function rrow(s){const r=s.results;return '<div class="card" style="padding:10px"
function resultsTab(){
 const r=D.stocks.filter(s=>s.results&&s.tier!=='-');
 let h='<div class="card"><div class="theme">How each shortlisted company did in
 ['Excellent','Good','Mixed','Weak','Loss-making'].forEach(v=>{const x=r.filter(s
  if(x.length)h+='<h2 class="'+rcol(v)+'">'+v+' ('+x.length+')</h2>'+x.slice(0,25
 return h;}
let U=null,sq='',sel='',thSel='',thFilter='ALL',thShow=60;
const ccls=v=>v==='BUY'?'g':v==='HOLD'?'b':v==='WAIT'?'a':'r';
const mc=x=>x==null?'-':Math.round(x).toLocaleString('en-IN');
const chips=a=>(a||[]).map(x=>'<span class="chip">'+esc(x)+'</span>').join('');
function loadUniv(){if(U)return Promise.resolve(U);if(D.univ){U=D.univ;return Pro
function getStock(sym){const f=D.stocks.find(x=>x.sym===sym);if(f)return Promise.
function openP(sym){tab='search';sel=sym;nav();draw();scrollTo(0,0);}
const urow=u=>'<div class="card" style="padding:10px" onclick="openP(\''+esc(u.s)
function themesTab(){
 const ar=D.areas||[];
 return '<div class="card"><div class="theme">Every scanned company grouped by ar
let mbShow=30;
function mbBlock(s){const m=s.mb;if(!m||!m.gates)return '';const ins=s.insider||[
function mbTab(){return '<div class="card"><div class="theme">Early multibagger h
function renderMb(){
 const el=document.getElementById('mbbox');if(!el)return;
 if(!U){el.innerHTML='<div class="empty">Loading all stocks...</div>';loadUniv().
 const all=U.filter(u=>u.mb!=null).sort((a,b)=>b.mb-a.mb);
 const buy=all.filter(u=>u.mbc==='MULTIBAGGER BUY'),watch=all.filter(u=>u.mbc==='
 const row=(u,i)=>'<div class="card" style="padding:10px;cursor:pointer" onclick=
 let h='<div class="stats" style="grid-template-columns:repeat(3,1fr)"><div class
 h+='<h2 class="g">Multibagger BUY ('+buy.length+')</h2>'+(buy.length?buy.slice(0
 h+='<h2 class="a">Strong watch ('+watch.length+')</h2>'+(watch.length?watch.slic
 if(!buy.length&&!watch.length)h+='<h2>Closest to qualifying</h2>'+all.slice(0,15
 el.innerHTML=h;}
```

```
let volMin=1.5,volF='GOOD';
function volTab(){const sel=(v,c)=>v==c?' selected':'';return '<div class="card">
function renderVol(){
 const el=document.getElementById('volbox');if(!el)return;
 if(!U){el.innerHTML='<div class="empty">Loading all stocks...</div>';loadUniv().
 let r=U.filter(u=>u.vr!=null&&u.vr>=volMin);
 if(volF==='UP')r=r.filter(u=>(u.d||0)>0);
 else if(volF==='GOOD')r=r.filter(u=>(u.d||0)>0&&(u.rv==='Excellent'||u.rv==='Goo
 else if(volF==='BUY')r=r.filter(u=>u.v==='BUY'||u.v==='HOLD');
 r.sort((a,b)=>b.vr-a.vr);
 const row=(u,i)=>'<div class="card" style="padding:10px;cursor:pointer" onclick=
 el.innerHTML='<div class="stats" style="grid-template-columns:repeat(2,1fr)"><di
let cmp=[],bestFilter='BUYHOLD',bestShow=30;
function bestTab(){return '<div class="card"><div class="theme">Every loaded stoc
function renderBest(){
 const el=document.getElementById('bestbox');if(!el)return;
 if(!U){el.innerHTML='<div class="empty">Loading all stocks...</div>';loadUniv().
 const f=bestFilter,tn={A:'5/5',B:'4/5',C:'3/5'};
 const rows=U.filter(u=>f==='ALL'||(f==='BUYHOLD'?(u.v==='BUY'||u.v==='HOLD'):f==
 let h='<div class="links" style="flex-wrap:wrap;margin:0 0 8px">'+[['BUYHOLD','B
 h+='<div class="theme" style="margin:0 2px 8px">'+rows.length+' of '+U.length+'
 if(cmp.length>=2)h+='<div class="links" style="margin:0 0 8px"><a href="#" style
 h+=rows.slice(0,bestShow).map((u,i)=>'<div class="card" style="padding:10px"><di
 if(rows.length>bestShow)h+='<div class="links"><a href="#" onclick="bestShow+=30
 el.innerHTML=h||'<div class="empty">Nothing here yet.</div>';}
function togCmp(s,on){if(on){if(!cmp.includes(s)){if(cmp.length>=3){alert('Pick u
function doCompare(){
 const el=document.getElementById('bestbox');el.innerHTML='<div class="empty">Loa
 Promise.all(cmp.map(getStock)).then(L=>{
  const row=(lab,fn)=>'<tr><td>'+lab+'</td>'+L.map(s=>{const v=fn(s);return '<td>
  const rs=s=>s.results||{},mg=s=>s.margin||{},pl=s=>s.plan||{};
  el.innerHTML='<div class="links" style="margin:0 0 8px"><a href="#" onclick="re
  row('Call',s=>'<b class="'+ccls(s.call)+'">'+esc(s.call)+'</b>')+row('Score /10
  row('Latest quarter',s=>esc(rs(s).verdict||''))+row('Sales YoY',s=>pc(rs(s).sal
  row('Cost quality',s=>esc(mg(s).verdict||''))+row('Operating profit YoY',s=>pc(
  row('ADX',s=>s.adx==null?null:Math.round(s.adx))+row('Rel. strength',s=>s.rs_ra
  row('Volume vs 20-day avg',s=>s.vol_ratio==null?null:s.vol_ratio.toFixed(1)+'x'
  row('Debtor days',s=>s.wc&&s.wc.debtor_days!=null?Math.round(s.wc.debtor_days):
  row('Order book',s=>s.ob_text?esc(s.ob_text):null)+row('Concall tone',s=>(s.con
  row('Institutions',s=>(s.holding||{}).inst?esc(s.holding.inst):null)+row('Price
  '</table></div></div><div class="theme">The stock with the higher score and mor
function renderThemes(){
 const el=document.getElementById('thres');if(!el)return;
 if(!U){loadUniv().then(renderThemes);return;}
 if(!thSel){el.innerHTML='<div class="empty">Choose an area above.</div>';return;
 const rank={BUY:0,WAIT:1,AVOID:2};
```

```
 const all_=U.filter(u=>(u.a||[]).includes(thSel)||u.t===thSel),fl=thFilter;
 const rows=all_.filter(u=>fl==='ALL'||(fl==='RULES'?u.tr!=='-':u.v===fl)).sort((
 const cy=D.sectors.find(c=>c.theme===thSel);
 let h=cy?'<div class="card"><div class="row"><b>Sector cycle</b><span class="bad
 h+='<div class="links" style="flex-wrap:wrap;margin:0 0 8px">'+['ALL','BUY','WAI
 h+=rows.slice(0,thShow).map(urow).join('')||'<div class="empty">No stocks.</div>
 if(rows.length>thShow)h+='<div class="links"><a href="#" onclick="thShow+=60;ren
 el.innerHTML=h;}
function searchTab(){return '<div class="card"><div class="theme">Search any scan
function sIn(v){sq=v;sel='';sres();}
function sres(){
 const el=document.getElementById('sres');if(!el)return;
 if(sel){el.innerHTML='<div class="empty">Loading...</div>';getStock(sel).then(s=
 const t=sq.trim().toLowerCase();
 if(t.length<2){el.innerHTML='<div class="empty">Type at least 2 letters.</div>';
 if(!U){loadUniv().then(sres);return;}
 const r=U.filter(x=>x.n.toLowerCase().includes(t)||x.s.toLowerCase().includes(t)
 el.innerHTML=r.length?r.map(urow).join(''):'<div class="empty">Not in the scanne
function htable(h){
 if(!h||!h.rows)return '<div class="theme">Shareholding data not available.</div>
 const nm={promoters:'Promoters',fiis:'FIIs (foreign)',diis:'DIIs (mutual funds,
 let t='<div style="overflow-x:auto"><table class="ht"><tr><th></th>'+h.labels.ma
 Object.keys(h.rows).forEach(k=>{const v=h.rows[k];t+='<tr><td>'+nm[k]+' %</td>'+
 if(h.holders&&h.holders.length)t+='<tr><td>Shareholders</td>'+h.holders.map(x=>'
 return t+'</table></div>';}
const bchips=n=>(n.who&&n.who.length?'<span class="chip">'+esc(n.who.join(', '))+
function profile(s){
 if(!s)return '<div class="empty">Could not load this stock. It may not have been
 const h=s.holding,r=s.results,inv=s.invest||[],fl=s.filings||[],k=s.concall||{},
 let o=(s.tier==='-'?'<div class="card" style="padding:10px"><div class="theme">T
 o+='<div class="card"><div class="sec">What it does</div><div style="font-size:1
 o+='<div class="card"><div class="sec">Concall: what management says and where i
 if(k.tone){o+='<div class="theme">Tone: <b>'+esc(k.tone)+'</b>'+(k.link?' | '+lk
 else o+='<div class="theme">No concall transcript read yet. It may not exist, or
 o+='</div>';
 o+='<div class="card"><div class="sec">Who gives it orders / partners</div>'+(cu
 o+='<div class="card"><div class="sec">What brokerages say (Kotak, ICICI Securit
 o+='<div class="card"><div class="sec">Quarterly results</div>';
 if(r&&r.hist){const cell=x=>'<td>'+(x==null?'-':Math.round(x).toLocaleString('en
  o+='<div style="overflow-x:auto"><table class="ht"><tr><th></th>'+r.hist.map(x=
 }else o+='<div class="theme">Not available.</div>';
 o+='</div><div class="card"><div class="sec">Who owns it (last 8 quarters)</div>
 o+='<div class="card"><div class="sec">Big investors, bulk/block deals, preferen
 o+='<div class="card"><div class="sec">Company filings: preferential issues, war
 return o;}
function ordersTab(){
```

```
 const O=D.orders||[];
 return '<div class="card"><div class="theme">Which customers are giving orders t
function bigTab(){
 const B=D.big||{acc:[],deals:[],prefs:[]};
 const nw=n=>'<a class="nw p" href="'+esc(n.link)+'" target="_blank" rel="noopene
 let h='<div class="card"><div class="theme">Where big money is going: institutio
 h+='<h2 class="g">Institutions accumulating</h2>'+(B.acc.length?B.acc.map(x=>'<d
 h+='<h2>Block / bulk deals and stake news</h2>'+(B.deals.length?'<div class="car
 h+='<h2 class="a">Preferential issues and fund raises</h2>'+(B.prefs.length?'<di
 return h;}
function brokersTab(){
 const R=D.brokers||[];const H={};
 R.forEach(n=>{(n.who&&n.who.length?n.who:['Other / unnamed']).forEach(w=>{(H[w]=
 const hs=Object.keys(H).sort((a,b)=>H[b].length-H[a].length);
 let h='<div class="card"><div class="theme">Where the big brokerages are buildin
 h+=hs.map(w=>'<h2>'+esc(w)+' ('+H[w].length+')</h2>'+H[w].slice(0,12).map(n=>'<d
 return R.length?h:'<div class="empty">No brokerage headlines found yet.</div>';}
function draw(){const a=document.getElementById('app');a.innerHTML=({picks,best:b
nav();draw();
</script></body></html>
"""

# --------------------------------------------------------------------------
if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--mode", choices=["full", "live", "demo", "catchup"], defaul
    a = ap.parse_args()
    if a.mode == "demo":
        d = demo_data()
        raw, state = {s["sym"]: s for s in d["stocks"]}, {k: v for k, v in d.item
    elif a.mode in ("full", "catchup"):
        if a.mode == "catchup":
            st_, rw_ = load_state(), load_raw()
            if st_.get("universe_n") and len(rw_) >= 0.97 * st_["universe_n"]:
                print("All listed stocks are already loaded - nothing to catch up
                sys.exit(0)
        raw, state = full_scan()
    else:
        raw, state = live_refresh()
    render(raw, state)
```